



Chapter 4

Complexity Analysis

4.1 Asymptotic Runtime

In the field of computer science, there is a lot of focus on the *efficiency* of algorithms. However, what exactly is efficiency, and how do we measure it? Upon first glance, it might seem appropriate to measure the efficiency of an algorithm by measuring its runtime. Suppose that two students are given two algorithms, A and B, that do the exact same thing, and they want to determine which algorithm is more time-efficient. Student 1 prepares algorithm A on their machine, and student 2 prepares algorithm B on their machine. Both students begin running their algorithms at the same time, and whichever algorithm finishes first is the one that is deemed more efficient.

However, this approach does not work because there are *too many confounding factors* that could affect the runtime of the algorithms. What if student 2 had a faster computer than student 1? Even if algorithm A was truly better, the fact that student 2 had a faster machine may lead us to incorrectly conclude that algorithm B is superior.

What if we remove this confounding variable of machine performance by limiting the experiment to a single machine? Suppose student 1 runs both algorithm A and algorithm B on their machine, one after another, and uses the runtime results to determine which algorithm is better. While this approach removes the discrepancy across machines, it is still not perfect. What if more intensive processes were running in the background when the student was running function A? There is still a chance that the inferior algorithm can come up on top.

Thus, while it may seem intuitive to base the time-efficiency of a program off its runtime, *pure runtime is actually an imperfect metric!* This is because the runtime of an algorithm can be confounded by factors other than the true efficiency of the algorithm itself. Variables such as CPU speed, the compiler and compiler flags used, and other programs running in parallel can influence the runtime of a program, even if these factors are not relevant to the algorithm itself. As a result, simply using program runtime to measure efficiency does not fully work! We want to measure efficiency in a way that removes these confounds and better reflects the efficiency of a program on its own.

To do this, we will be using **input size** instead of actual runtime to determine the relative efficiency of a program. That is, instead of asking ourselves *"How long does this program take to run?"* when measuring efficiency, we ask ourselves *"How does the program's runtime scale with input size?"*. If we double the size of the input, does the runtime also double? Does it quadruple? Or does it stay the same? Given two algorithms that perform the same task, if the runtime of one algorithm grows much slower than the other as input size increases, then that algorithm is considered to be better than the other when it comes to time efficiency.

Why is this approach better? As mentioned before, the actual runtime of a program may be confounded by factors such as CPU speed, compiler flags, and processes running the background. *However, for input sizes that are large enough, the rate of growth of runtime with respect to input size is independent of most external factors!* Regardless of how fast the CPU is, which compiler flags are used, or what programs are running in the background, the rate of growth of an algorithm should stay relatively consistent as you increase its input size. This idea forms the basis of **asymptotic runtime**, which describes how the runtime of an algorithm grows as the size of its input grows. We use asymptotic runtime to measure and compare the time-efficiency of algorithms.

Asymptotic runtime and actual runtime are not the same thing! For example, if program A completes in 2.81 seconds, the number 2.81s represents the *runtime* of the program in seconds. However, if program A's runtime doubles when input size doubles, triples when input size triples, and quadruples when input size quadruples, etc., we say there is a linear relationship between the input size and the runtime of the program. This linear relationship describes the *asymptotic runtime* of the program. Unlike runtime, which is expressed as a unit of time (e.g., seconds), the asymptotic runtime of an algorithm describes the relationship between input size and runtime, and it is expressed using something known as **big-O notation**. This will be covered in the next section.

4.2 Complexity Classes and Big-O Notation

In the previous section, we introduced the concept of asymptotic runtime. In this section, we will go over how the asymptotic runtime of an algorithm can be computed. To do so, we will first need to conceptualize how we want to measure our input size.

What counts as "one unit" of input? Ultimately, it depends on what the algorithm is designed to do: you want to select a unit of measurement that is related to the functionality of your algorithm. For example, if your algorithm sorts an array of integers, it would make sense to treat the input size as the number of integers in the array.

On a similar vein, if you had a graph with V vertices and E edges, the way you define the input size should be related to what your algorithm does. If your algorithm only deals with the number of vertices in the graph (and ignores the edges), it would be sufficient to define input size in terms of the number of vertices V . However, if your algorithm does work on *both* the vertices and edges of the graph, it may be better to define input size in terms of both V and E . The more information you include when defining your input size, the more information you can potentially get out from time complexity analysis (for example, measuring asymptotic runtime using both V and E can give you an idea of which variable has a greater impact on the performance of an algorithm).

After deciding what counts as a unit of input, you can vary the amount of input you feed into the algorithm, measure the amount of time it takes to run the algorithm, and plot out the points on a graph to visualize a relationship between input size and runtime. *To ensure that this relationship can be clearly determined by the results, make sure that the input sizes you experiment with are large enough!* If the input size is too small, your results may be obfuscated by variance, and you may not be able to clearly see how runtime scales with the input size.

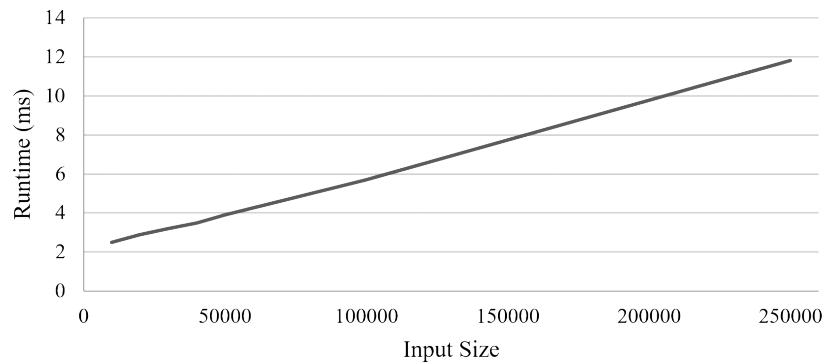
Example 4.1 Consider the following algorithm, which sums up all the numbers in an array of integers. Determine the asymptotic runtime complexity of this algorithm by measuring runtime with respect to input size.

```
1  int32_t sum_array(int32_t arr[], size_t size) {
2      int32_t total = 0;
3      for (size_t i = 0; i < size; ++i) {
4          total += arr[i];
5      } // for i
6      return total;
7  } // sum_array()
```

Our first step is to determine how we want to measure our input size. Since we are summing an array, it would make sense for input size to be the size of the input array. Now that we know our unit of input, lets measure the runtime of the program with different input sizes. The following data was collected by running the `sum_array()` function for array sizes of 10000, 20000, 30000, 40000, 50000, 100000, and 250000.

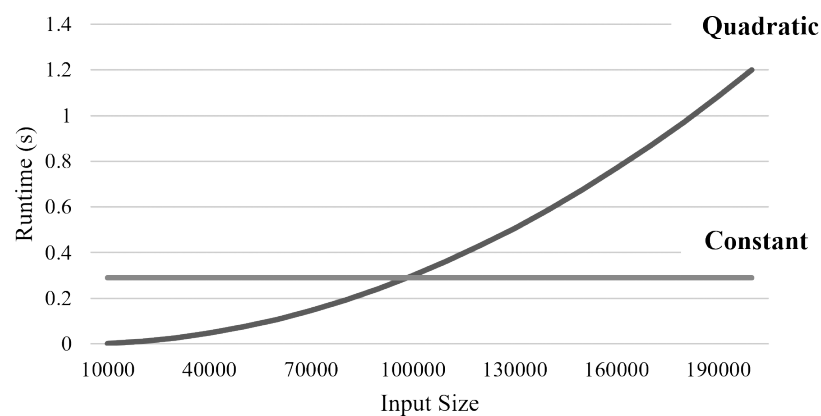
Input Size	Runtime (ms)
10000	2.5
20000	2.9
30000	3.2
40000	3.5
50000	3.9
100000	5.7
250000	11.8

Graphing this out, we can see that there is a *linear* relationship between the input size and the runtime. That is, as the size of the input size increases, the runtime increases linearly (in a straight line).



The **time complexity** of an algorithm is a way to express the relationship between runtime and input size. If there is a linear relationship between runtime and input size, then the algorithm has a *linear time complexity*.

Linear time is just one of many time complexity classes that an algorithm can exhibit. If there exists a quadratic relationship between input size and runtime, the algorithm has a *quadratic time complexity*. If the runtime stays relatively constant regardless of the input size, then the algorithm has a *constant time complexity*. Examples of these two time complexity classes are shown below:



Big-O notation can be used to express these time complexities mathematically. Instead of describing time complexity classes using words like constant, linear, and quadratic, we describe them using a notation that follows the form $O(f(\dots))$, where $f(\dots)$ is a function that depends on an algorithm's input size variables.

Let's revisit the graph of the `sum_array()` function from example 4.1. How can we express its asymptotic runtime using big-O notation? To do so, we would need to find $f(n)$, where n is the size of the array we want to sum. We can determine $f(n)$ by following these steps:

1. Express the runtime as a function of the input size variables.
2. Identify the fastest growing terms in the equation calculated in step 1 and drop all lower order terms.
3. Drop the coefficients of the fastest growing terms.

After doing this, you will be left with the term that is used for the big-O notation. Let's try an example:

Example 4.2 Express the time complexity class of the `sum_array()` function using big-O notation.

To do this, we can follow the three steps above.

1. **Express the runtime as a function of the input size variables.** We can use the points shown on the previous page to determine the line of best fit. By doing so, we can express the runtime $T(n)$ as the function of the input size n using the equation:

$$T(n) = (4 \times 10^{-5})n + 2$$

2. **Identify the fastest growing terms in the equation calculated in step 1 (and drop all lower order terms).** The fastest growing term is the term that grows the fastest as the input size (in this case, n) increases without bound. In the equation above, the constant term 2 does not grow at all as n increases, while the term $(4 \times 10^{-5})n$ grows linearly as n increases. The fastest growing term is thus $4 \times 10^{-5}n$.

In many cases, the fastest growing term is the term with the largest exponent. For example, if the relationship between input size and runtime were $T(n) = 5n^3 + 2n^7 + 16n^6 + 9n^2 + 55n^4 + 23$, the fastest growing term would be $2n^7$, since this is the term with the largest exponent, 7. See the following page for orders of growth.

3. **Drop the coefficients of the fastest growing terms.** Since the fastest growing term is $4 \times 10^{-5}n$, we can remove the coefficient (4×10^{-5}) . We are left with the term n , which is our value of $f(n)$.

The time complexity of `sum_array()` is therefore $O(n)$.

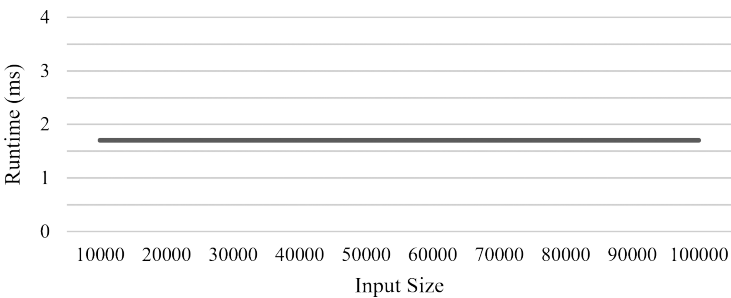
Example 4.3 You are given the following function. Express the time complexity of the `return_zero()` function using big-O notation:

```
1  int32_t return_zero(int32_t arr[], size_t size) {
2      int32_t zero = 0;
3      return zero;
4  } // return_zero()
```

After running this function with arrays of many different sizes and measuring its runtime, we get the following:

Input Size	Runtime (ms)
10000	1.7
25000	1.7
50000	1.7
75000	1.7
100000	1.7

The runtime of the function is plotted below:



We can use this information to follow the three steps on the previous page.

- Express the runtime as a function of the input size variables.** We can express the runtime $T(n)$ as a function of the input size variable n using the equation:
$$T(n) = 1.7$$
- Identify the fastest growing terms in the equation calculated in step 1 (and drop all lower order terms).** There is only one term here (1.7), so it is by default the fastest growing term.
- Drop the coefficients of the fastest growing terms.** Although it may not be obvious, the term 1.7 can actually be written as $1.7n^0$. Thus, the coefficient of the fastest growing term is 1.7 — removing this, we end up with just n^0 , or 1.

The time complexity of `return_zero()` is therefore $O(1)$.

Why are we allowed to drop coefficients and lower order terms when dealing with time complexities? Recall that the time complexity of an algorithm only deals with how its runtime *scales* with respect to input size. Thus, it does not matter what the coefficients and lower order terms are, as they are not directly responsible for an algorithm’s asymptotic rate of growth. For example, the functions $T(n) = 0.5n^2 + 13n + 5$ and $T(n) = 14n^2 + 3n + 9$ both experience the same quadratic growth with respect to input size, even though they have different coefficients and lower order terms; it would therefore be more meaningful to express both functions with a complexity of $O(n^2)$.

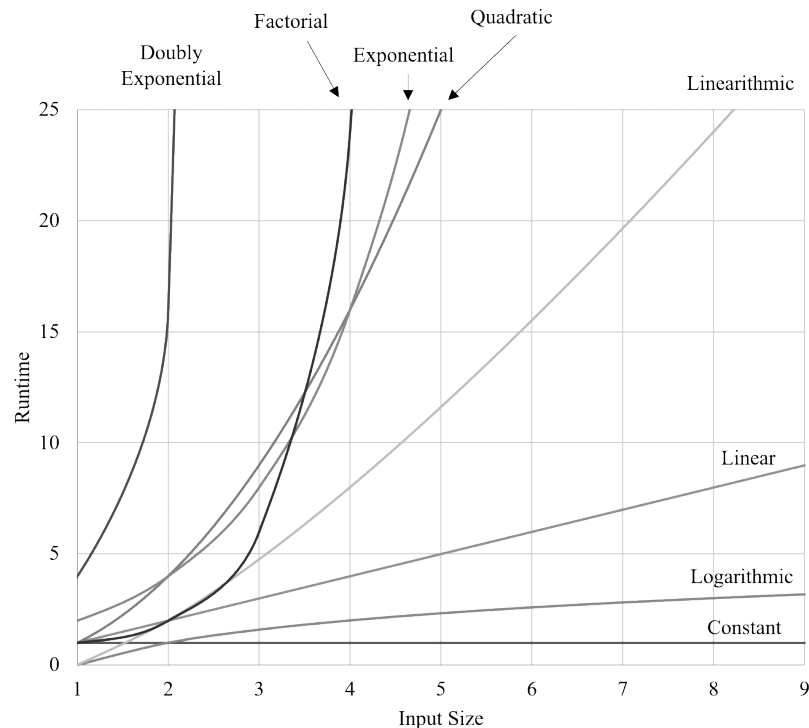
Remark: Just because coefficients are dropped does *not* mean that they are not important! An algorithm whose runtime can be expressed using the function $T(n) = 999n^2$ is slower than an algorithm whose runtime can be expressed using $T(n) = 0.01n^3$ for most reasonable input sizes n , even if it technically has a better time complexity.

This is where knowing the difference between runtime and asymptotic runtime is crucial. When dealing with time complexities, we only work with the rate of growth and not the actual runtime itself. Because of this, *a better time complexity does not guarantee a faster runtime*. As shown with the example above, it is entirely possible for an algorithm with a better asymptotic time complexity to run slower — this is because time complexity only indicates how fast runtime *scales with input size*, not the actual runtime itself!

The following table lists a few common complexity classes, in order of rate of growth:

Name of Complexity Class	Big-O Notation
Constant	$O(1)$
Logarithmic	$O(\log(n))$
Linear	$O(n)$
Loglinear/Linearithmic	$O(n \log(n))$
Quadratic	$O(n^2)$
Cubic	$O(n^3)$
Polynomial (includes quadratic and cubic)	$O(n^d)$ where d is a constant
Exponential	$O(c^n)$ where c is a constant
Factorial	$O(n!)$
Doubly Exponential	$O(2^{2^n})$

A visualization of each of these complexity classes is shown below:



To summarize this section so far, we introduced the concept of big-O notation and how we can derive the time complexity of an algorithm using experimentation. However, if the chapter ended here, it would be incomplete. First of all, experimentation is not always feasible (for example, you will not be able to run code on an exam). And, in the cases where experimentation can be done, it is not always easy. Even with data, how can we derive an equation that best relates runtime to input size? How large will our input size have to be to derive a precise estimation that is not subject to noise and variation? Furthermore, if we have multiple variables at play (such as the number of vertices (V) and edges (E) in a graph), how can we take all of these variables into account and distinguish among them in our measurements?

The good news is that you do not need a computer or any experimentation to determine the time complexity of an algorithm; simply looking at the code itself is often more than enough to calculate its time complexity. This can be done by analyzing the steps that the algorithm takes and measuring how the *number of steps taken* (rather than its runtime) scales with the input size.

4.3 Measuring Complexity by Counting Steps

In many cases, you will not be able to measure the runtime of an algorithm via experimentation. Luckily, experimentation is not the only way to calculate an algorithm's time complexity. An alternative approach is to count the *number of steps* that an algorithm takes and express the result as a function of the input size. This works because the number of steps that an algorithm takes is directly related to its runtime — as long as the runtime of a single step is independent of input size and can be done in constant time, a function that expresses the relationship between number of steps and input size should have the *same order of growth* as a function that expresses the relationship between runtime and input size.

For example, suppose that there is a linear relationship between the number of steps an algorithm takes and its input size. In this case, increasing the input size causes a linear growth in the step count needed to complete the algorithm. Since each step takes constant time, increasing the step count linearly would also increase the runtime linearly. Similarly, if there is a quadratic relationship between the number of steps that an algorithm takes and its input size, a quadratic blowup in the number of steps would also lead to a quadratic blowup in runtime.

What counts as a step in a program? In this class, we will consider the following operations as a single step. These operations all take a constant amount of time and are *not* dependent on the size of the input. For example, the time it takes to multiply two fixed-size numbers in an array should remain roughly the same regardless of whether the input array has a size of ten or ten million.¹

1. variable assignment (e.g. $a = 5$)
2. arithmetic operation (e.g. $a + b$)
3. comparison operation (e.g. $a < b$)
4. array indexing (e.g. $\text{arr}[i]$)
5. pointer reference (e.g. *ptr)
6. function call
7. function return

With this in mind, we can use the number of steps an algorithm takes to determine its time complexity. The following procedure can be used to complete this process (this is very similar to the procedure outlined in section 4.2):

1. Express the *step count* as a function of the input size variables.
2. Identify the fastest growing terms in the equation calculated in step 1 and drop all lower order terms.
3. Drop the coefficients of the fastest growing terms.

¹For certain algorithms, it is entirely possible that you may want to measure input size in terms of the number of bits or digits in an integer, for which claiming arithmetic as an "constant" time operation may be an oversimplification, but we will not worry about that complication here.

Let's consider the example in the previous section, this time using a step-counting approach:

Example 4.4 Express the time complexity of the `sum_array()` function using big-O notation. Let input size n be the size of the array.

```
1  int32_t sum_array(int32_t arr[], size_t size) {
2      int32_t total = 0;
3      for (size_t i = 0; i < size; ++i) {
4          total += arr[i];
5      } // for i
6      return total;
7  } // sum_array()
```

In this example, we will count the number of steps needed to complete this function. The input size n is the value of the variable `size`, so we will use both `size` and n interchangeably. Let's look at the function line-by-line and analyze the number of steps required.

- On line 1, we have the function call definition. For our examples, we will ignore this line and attribute the work required to call the function to the caller of the `sum_array()` function instead of the `sum_array()` function itself. In other words, if `main()` calls `sum_array()`, then line 1 executes — however, the work for line 1 will be counted for `main()`, and not `sum_array()`.
- Line 2 is where the implementation of `sum_array()` begins. We have a variable assignment, which takes one step.
- Lines 3 and 4 contain a `for` loop. Line 3 itself has an initialization assignment (`i = 0`) which is done once, a comparison that is done $n + 1$ times (one comparison for every time the body of the loop runs, plus one additional comparison for the final check to determine that the loop should end), and an arithmetic operation (`++i`) that is run n times (once for each time the body of the loop runs).
- Line 4 contains an arithmetic operation, which takes one step. However, since this is run inside a `for` loop that runs n times, the total number of steps taken is n .
- Line 6 returns from the function, which takes one step.

With this information, we can express the number of steps as a function of input size.

1. **Express the step count as a function of the input size variables.** The total number of steps taken is:

- 1 step on line 2
- $1 + (n + 1) + n$ steps on line 3
- n steps on line 4
- 1 step on line 6

which totals to $1 + 1 + n + 1 + n + n + 1 = 3n + 4$ steps.

Thus, our equation that expresses the step count $S(n)$ as a function of input size n is: $S(n) = 3n + 4$

2. **Identify the fastest growing terms in the equation calculated in step 1 (and drop all lower order terms).** The fastest growing term in the expression is $3n$.
3. **Drop the coefficients of the fastest growing terms.** Removing the coefficient from $3n$ leaves us with n .

The time complexity of `sum_array()` is therefore $O(n)$.

We got the same answer that we did when we measured the complexity using the more tedious method of experimentation. However, we are still doing more work than necessary. For example, consider the `+=` operator on line 4. In the example, we listed it as one step. However, the `+=` operator actually involves two separate steps: an addition followed by an assignment. A similar argument can be made for the `++i` operation on line 3: the `++` operator first increments, then assigns, resulting in two steps rather than one. Thus, the equation $3n + 4$ is not fully accurate.

So, why did we provide an example with an "inaccurate" analysis? It turns out that debating whether `+=` counts as one step or two steps does not matter! Since coefficients and lower order terms are removed anyway, small differences like these will not affect the time complexity class. If we had treated `+=` and `++` as two steps instead of one in the previous example, we would have gotten the equation $S(n) = 5n + 4$. However, the growth of this function is still linear! The difference between the 3 and the 5 does not change the overall time complexity class for the algorithm. This little detail actually makes our job easier, since it does not matter if a line of code takes 1, 2, 3, or even 999 steps; as long as the number is a constant, the difference in steps does not affect the answer. Because these details do not matter, we do not need to calculate the exact number of steps when deriving the complexity class of an algorithm. Instead, it would suffice to just determine the *asymptotic complexity* of each line instead — any coefficients or lower order terms would eventually be dropped regardless. This leads us to the following procedure for determining the time complexity of an algorithm, given its code:

Calculating the Time Complexity of a Function By Counting Steps

1. Start at the first line of the function, not including the function header. Walk through the code line by line, assigning each line with a value using the following rules (it may be easy to jot it down to the side).
 - a. If the line is the definition of a loop (e.g. `for(...)` or `while(...)`, etc.), assign it with the number of times the loop executes (you may ignore coefficients and lower order terms).
 - b. If the line is not a loop definition, assign it with the time complexity required to execute the line.
2. After all lines have been assigned a value, identify the loops in the function. For non-nested loops, *add* all time complexities in the body of the loop, and then *multiply* this result with the number of times the loop runs. That is, to determine the overall time complexity of a loop, multiply the time complexity of its body with the number of times the loop runs.
3. For nested loops (loops within a loop), start from the innermost loop and work outwards, calculating the total complexity at each step. This can be done by starting with the innermost loop and running the procedure from step 2 on that loop. Then, use the value you get to repeat step 2 on the next innermost loop, repeating until you reach the outermost loop.
4. After all the loops are taken care of, *add* the remaining complexities to get the final time complexity. Remember to remove coefficients and lower order terms.

Let's try a few examples.

Example 4.5 Express the time complexity of the `sum_array()` function using big-O notation. Let input size n be the size of the array.

```
1  int32_t sum_array(int32_t arr[], size_t size) {
2      int32_t total = 0;
3      for (size_t i = 0; i < size; ++i) {
4          total += arr[i];
5      } // for i
6      return total;
7  } // sum_array()
```

To approach this problem, walk through each line of the function code and assign it with the time required to execute it:

- **Line 2:** this is an assignment, which takes constant time, so line 2 gets assigned $O(1)$.
- **Line 3:** this is a loop definition, so we assign it with the number of times it runs. This loop runs `size` times. Since `size` is the input size n , we can substitute `size` with n , so line 3 gets assigned $O(n)$.
- **Line 4:** this is an addition and an assignment, which takes constant time, so line 4 gets assigned $O(1)$.
- **Line 6:** this is a return statement, which takes constant time, so line 5 gets assigned $O(1)$.

At this point, we should have the following (the box represents the scope of the for loop):

1	<code>int32_t sum_array(int32_t arr[], size_t size) {</code>	1	
2	<code>int32_t total = 0;</code>	2	$O(1)$
3	<code>for (size_t i = 0; i < size; ++i) {</code>	3	$O(n)$:
4	<code>total += arr[i];</code>	4	$O(1)$
5	<code>} // for i</code>	5	
6	<code>return total;</code>	6	$O(1)$
7	<code>} // sum_array()</code>	7	

Now that we have listed out all the time complexities, we will now combine the loops. Here, we only have one loop defined on line 3. To determine the overall time complexity of running this loop, we can multiply the body of the loop (which takes $O(1)$ time, as indicated on line 4) with the number of times the loop runs: $O(n) \times O(1) = O(n)$.

1	<code>int32_t sum_array(int32_t arr[], size_t size) {</code>	1		
2	<code>int32_t total = 0;</code>	2	$O(1)$	
3	<code>for (size_t i = 0; i < size; ++i) {</code>	3	$O(n)$	the for loop runs a $O(1)$ operation $O(n)$ times, so the overall time complexity of the loop is $O(n) \times O(1)$, or $O(n)$.
4	<code>total += arr[i];</code>	4		
5	<code>} // for i</code>	5		
6	<code>return total;</code>	6	$O(1)$	
7	<code>} // sum_array()</code>	7		

Now that we have dealt with all the loops, we can sum up the remaining time complexities to get the overall time complexity of the function:

$$T(n) = O(1) + O(n) + O(1)$$

The highest order term is $O(n)$, so the time complexity of the `sum_array()` function is $O(n)$.

Example 4.6 Express the time complexity of the `foo()` function using big-O notation. Let the input size n be the size of the array. The `sum_array()` function is the same as the one in the previous example.

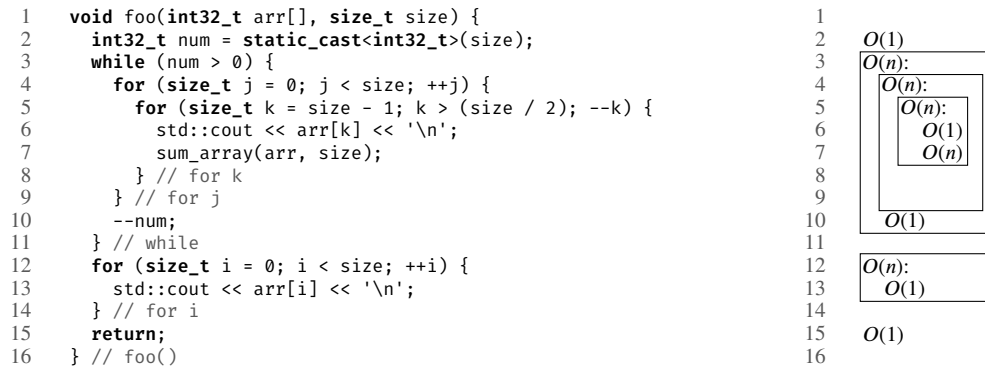
```
1  void foo(int32_t arr[], size_t size) {
2      int32_t num = static_cast<int32_t>(size);
3      while (num > 0) {
4          for (size_t j = 0; j < size; ++j) {
5              for (size_t k = size - 1; k > (size / 2); --k) {
6                  std::cout << arr[k] << '\n';
7                  sum_array(arr, size);
8              } // for k
9          } // for j
10         --num;
11     } // while
12     for (size_t i = 0; i < size; ++i) {
13         std::cout << arr[i] << '\n';
14     } // for i
15     return;
16 } // foo()
```

Let's walk through each line of this function and assign it with the time required to execute the line:

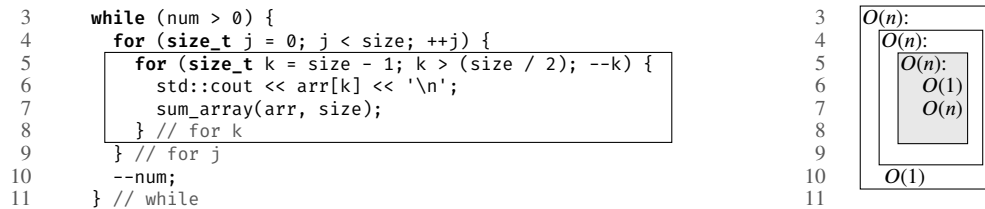
- **Line 2:** this is an assignment, which takes constant time, so line 2 gets assigned $O(1)$.
- **Line 3:** this is a loop definition, so we assign it with the number of times it runs. Notice how this loop runs as long as `num` is greater than 0, where `num` is initialized to `size` and is decremented after each run of the loop. Thus, the while loop runs `size`, or n times.
- **Line 4:** this is a loop definition, so we assign it with the number of times it runs. Because this loop runs from 0 to `size` and increments by 1 per iteration, it runs n times.
- **Line 5:** this is a loop definition, so we assign it with the number of times it runs. This loop runs from $n - 1$ to $n/2$, or roughly $n/2$ times. This can be represented using the complexity class $O(n)$.
- **Line 6:** this is a print statement with array indexing — the runtime of this operation is independent of input size, so this line gets assigned constant time, or $O(1)$.
- **Line 7:** the `sum_array()` function runs in linear $O(n)$ time, from the previous example. Thus, this line gets assigned with $O(n)$.
- **Line 10:** this is a constant time operation, so it gets assigned $O(1)$.
- **Line 12:** this is a loop definition, so we assign it with the number of times it runs. This loop runs from 0 to `size` and increments by 1 per iteration, so it runs n times.

- **Line 13:** this is a constant time operation, so it gets assigned $O(1)$.
- **Line 15:** this is a constant time operation, so it gets assigned $O(1)$.

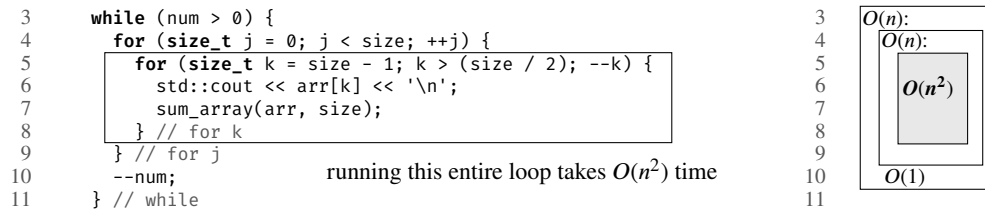
At this point, we have the following (the boxes represent the scope of the loops):



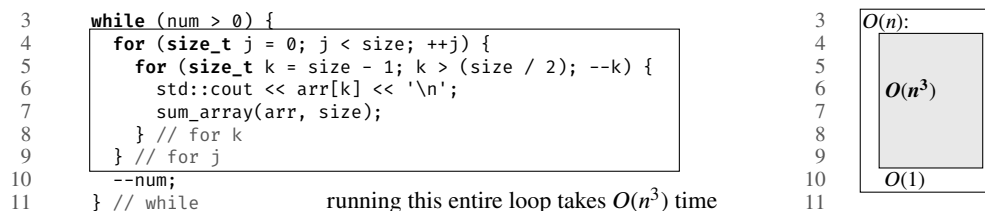
Now, we will walk through the code and condense the loops. The first loop we encounter is a triple nested loop spanning lines 3-11, consisting of one while loop and two for loops. Since this loop is nested, we start from the innermost loop and work outwards. We will first begin by analyzing the for loop defined on line 5 (the innermost loop):



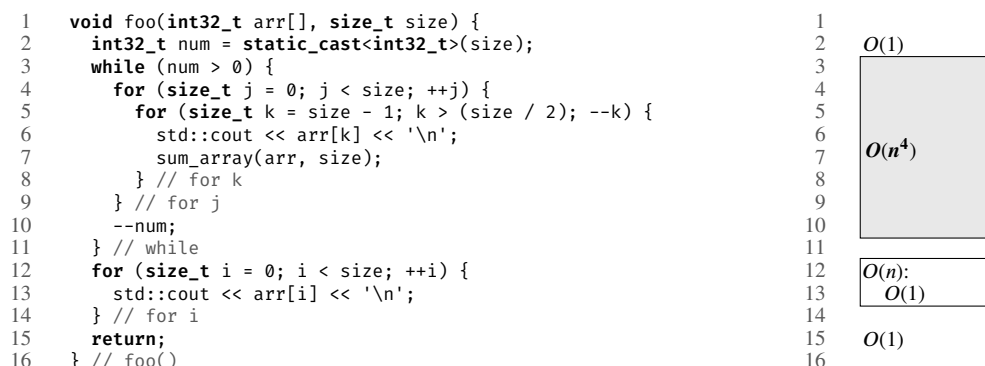
When dealing with a non-nested loop, we first compute the complexity class of the loop body (lines 6-7). The time complexity of lines 6 and 7 is $O(1) + O(n)$, which can be simplified to $O(n)$ after dropping lower order terms. Thus, the body of the loop (lines 6 and 7) runs in $O(n)$ time. Since lines 6-7 run $O(n)$ times inside the innermost for loop defined on line 5, the time complexity of the innermost loop is therefore $O(n) \times O(n) = O(n^2)$. After condensing this loop into a single complexity class $O(n^2)$, we get the following:



Now, we move on to the next innermost loop, defined on line 4. This loop runs n times, and the body of the loop runs in $O(n^2)$ time. Thus, the combined time complexity of the two loops is $O(n) \times O(n^2) = O(n^3)$. Lines 4-9 have now been condensed into a single time complexity class:

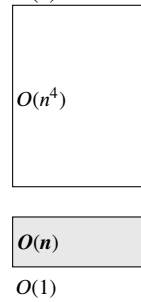


We have one loop left in this nested loop (the while loop defined on line 3). The body of this loop has an overall time complexity of $O(n^3) + O(1) = O(n^3)$, and it is run a total of $O(n)$ times. Thus, the overall time complexity of the while loop on line 3 is $O(n) \times O(n^3) = O(n^4)$. The nested loop on lines 3-11 has been condensed into a single time complexity of $O(n^4)$, so now we can move on to the next loop on line 12.



The loop on line 12 runs a $O(1)$ operation $O(n)$ times, so the complexity of this loop is $O(n) \times O(1) = O(n)$.

1	<code>void foo(int32_t arr[], size_t size) {</code>	1	
2	<code>int32_t num = static_cast<int32_t>(size);</code>	2	$O(1)$
3	<code>while (num > 0) {</code>	3	
4	<code>for (size_t j = 0; j < size; ++j) {</code>	4	
5	<code>for (size_t k = size - 1; k > (size / 2); --k) {</code>	5	
6	<code>std::cout << arr[k] << '\n';</code>	6	
7	<code>sum_array(arr, size);</code>	7	
8	<code>} // for k</code>	8	
9	<code>} // for j</code>	9	
10	<code>--num;</code>	10	
11	<code>} // while</code>	11	
12	<code>for (size_t i = 0; i < size; ++i) {</code>	12	
13	<code>std::cout << arr[i] << '\n';</code>	13	
14	<code>} // for i</code>	14	
15	<code>return;</code>	15	
16	<code>} // foo()</code>	16	



After dealing with all the loops, we have managed to condense this function into four sequential steps: an assignment that takes $O(1)$ time, a triple-nested while loop that runs in $O(n^4)$ time, and for loop that runs in $O(n)$ time, and a return statement that takes $O(1)$ time. Since these steps happen one after another, we can add their complexities together to obtain the overall time complexity of the `foo()` function: $O(1) + O(n^4) + O(n) + O(1) = O(n^4)$.

Note that lower order terms can be ignored, and that the time complexity of the entire `foo()` function is governed by the term with the largest time complexity class. Here, the nested loop that spans lines 3-11 runs in $O(n^4)$ time, which dominates all the other components of the function. As a result, this loop is a bottleneck that determines the rate of growth for the entire `foo()` function.

4.4 Logarithmic Runtime

※ 4.4.1 Analyzing Logarithmic Runtime

In the previous two examples, we dealt with loops that ran n iterations, where n is the input size. An example of such a loop is shown below.

```
for (int32_t i = 1; i < input_size; ++i) { ... }
```

However, not all loops behave like this. What if we changed the update condition so that the loop counter is *doubled* rather than incremented?

```
for (int32_t i = 1; i < input_size; i *= 2) { ... }
```

How many times does the loop run now? On the first iteration, the loop counter i is 1. Then, it becomes 2, then 4, then 8, then 16, doubling with each iteration until it reaches the value of `input_size`, or n . Thus, the number of times the loop runs is equal to the number of times we can double i before it hits `input_size`. In mathematical terms, this can be represented as:

$$2^k = n$$

where k represents the number of times the loop can run before i exceeds `input_size` or n . Solving for k , we get:

$$k = \log_2(n)$$

Thus, the second for loop (where i is doubled with each iteration) runs on the order of $\log_2(n)$ times, or $O(\log_2(n))$ times. For example, suppose the input size is 64, and the loop counter i doubles with each iteration. The loop would run six times: when i equals 1, 2, 4, 8, 16, and 32. After the sixth iteration runs, i gets doubled to 64, and the loop exits. This makes sense with our analysis, since $\log_2(64)$ is indeed 6. If the input size were a non-power of two like 65, the loop would run seven times, which is still on the order of $\log_2(n)$.

Division works similarly. Consider the following loop:

```
for (int32_t i = input_size; i > 1; i /= 2) { ... }
```

Here, the loop counter i is halved at each iteration of the loop. Now, we want to calculate the number of times we can halve the input size before we hit 1, since this is the number of times the loop runs. Once again, we can write this in mathematical terms as:

$$n \left(\frac{1}{2} \right)^k = 1$$

Solving for k , we can rewrite this as:

$$k = \log_2(n)$$

Division gives us the same result as multiplication. This leads us to an interesting conclusion: if multiplication or division is involved in the update of a loop, there is a good chance that a logarithmic term is in play. Furthermore, if the search space of a problem gets cut in half (or by some other factor) with each iteration, a logarithmic term is likely involved.

Here is an example of a logarithmic algorithm in practice: suppose you wanted to find someone in a phone book, where the phone book has a total of n pages. If you looked through every page of the phone book until you find the person you want, that would be an $O(n)$ algorithm. However, if the phone book is alphabetical by last name, you can use this fact to reduce the work you have to do! For example, if you wanted to find a last name starting with the letter "P", you could completely ignore the first half of the book since you know that the letter "P" must be in the second half. You can continue this process by flipping to the middle page of the second half and checking if "P" comes before or after. This eliminates half of the remaining pages at each step (similar to the $i /= 2$ update of the loop), leading to a logarithmic algorithm with complexity $O(\log_2(n))$. In other words, you would only need to check at most $O(\log_2(n))$ pages of a phone book with n pages to find any person you want!

Example 4.7 Express the time complexity of the `foo()` function using big-O notation. Let the input size n be the size of the array.

```
1  int32_t foo(int32_t arr[], size_t size) {
2      int32_t total = 0;
3      for (size_t i = 1; i < size; i *= 2) {
4          total += arr[i];
5      } // for i
6      return total;
7  } // foo()
```

Walking through the code and assigning each line with its complexity results in the following:

1	<code>int32_t foo(int32_t arr[], size_t size) {</code>	1	
2	<code>int32_t total = 0;</code>	2	$O(1)$
3	<code>for (size_t i = 1; i < size; i *= 2) {</code>	3	$O(\log_2(n))$:
4	<code>total += arr[i];</code>	4	$O(1)$
5	<code>} // for i</code>	5	
6	<code>return total;</code>	6	$O(1)$
7	<code>} // foo()</code>	7	

Here, the loop counter i is multiplied by 2 with each iteration, so the number of times the loop runs is $O(\log_2(n))$. Since the body of the loop runs in $O(1)$ time, the total time complexity of the for loop on line 3 is $O(\log_2(n)) \times O(1) = O(\log_2(n))$.

1	<code>int32_t foo(int32_t arr[], size_t size) {</code>	1	
2	<code>int32_t total = 0;</code>	2	$O(1)$
3	<code>for (size_t i = 1; i < size; i *= 2) {</code>	3	$O(\log_2(n))$
4	<code>total += arr[i];</code>	4	
5	<code>} // for i</code>	5	
6	<code>return total;</code>	6	$O(1)$
7	<code>} // foo()</code>	7	

The overall time complexity of the `foo()` function is thus $O(1) + O(\log_2(n)) + O(1)$, or simply $O(\log_2(n))$ after dropping lower order terms.

Example 4.8 Express the time complexity of the `foo()` function using big-O notation, in terms of the input value n .

```
1  void foo(int32_t n) {
2      for (int32_t i = 1; i < n; ++i) {
3          for (int32_t j = 1; j < log2(n); j *= 2) {
4              for (int32_t k = 1; k < n; k *= 2) {
5                  std::cout << "Potato!\n";
6              } // for k
7          } // for j
8      } // for i
9  } // foo()
```

As before, we will walk through each line of this function and assign it with the time required to execute the line:

- **Line 2:** this loop runs from 1 to n , incrementing the counter by one with each iteration, so it gets assigned with $O(n)$.
- **Line 3:** this loop runs from 1 to $\log_2(n)$, multiplying the counter by two with each iteration. Thus, the number of times this loop runs is equal to k in the following equation:

$$2^k = \log_2(n)$$

$$k = \log_2(\log_2(n))$$

- **Line 4:** this loop runs from 1 to n , multiplying the counter by two with each iteration. The number of times this loop runs is $O(\log_2(n))$.
- **Line 5:** this is a constant time operation, so it gets assigned $O(1)$.

1	<code>void foo(int32_t n) {</code>	1	
2	<code>for (int32_t i = 1; i < n; ++i) {</code>	2	$O(n)$:
3	<code>for (int32_t j = 1; j < log2(n); j *= 2) {</code>	3	$O(\log_2(\log_2(n)))$:
4	<code>for (int32_t k = 1; k < n; k *= 2) {</code>	4	$O(\log_2(n))$:
5	<code>std::cout << "Potato!\n";</code>	5	$O(1)$
6	<code>} // for k</code>	6	
7	<code>} // for j</code>	7	
8	<code>} // for i</code>	8	
9	<code>} // foo()</code>	9	

The innermost loop on line 4 runs a $O(1)$ operation $O(\log_2(n))$ times, so the time complexity of this loop is $O(\log_2(n)) \times O(1) = O(\log_2(n))$.

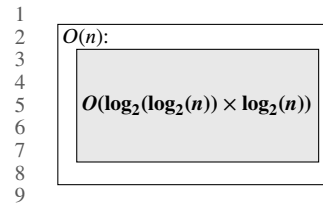
1	<code>void foo(int32_t n) {</code>	1	
2	<code>for (int32_t i = 1; i < n; ++i) {</code>	2	$O(n)$:
3	<code>for (int32_t j = 1; j < log2(n); j *= 2) {</code>	3	$O(\log_2(\log_2(n)))$:
4	<code>for (int32_t k = 1; k < n; k *= 2) {</code>	4	$O(\log_2(n))$
5	<code>std::cout << "Potato!\n";</code>	5	
6	<code>} // for k</code>	6	
7	<code>} // for j</code>	7	
8	<code>} // for i</code>	8	
9	<code>} // foo()</code>	9	

The loop on line 3 runs the loop on line 4 $O(\log_2(\log_2(n)))$ times, so its complexity is $O(\log_2(\log_2(n)) \times O(\log_2(n)) = O(\log_2(\log_2(n)) \times \log_2(n))$.

```

1 void foo(int32_t n) {
2   for (int32_t i = 1; i < n; ++i) {
3     for (int32_t j = 1; j < log2(n); j *= 2) {
4       for (int32_t k = 1; k < n; k *= 2) {
5         std::cout << "Potato!\n";
6       } // for k
7     } // for j
8   } // for i
9 } // foo()

```

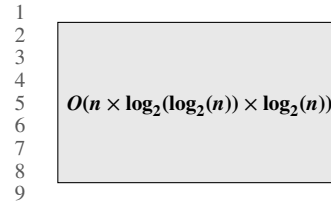


The loop on line 2 runs the loop on line 3 $O(n)$ times, so its complexity is $O(n) \times O(\log_2(\log_2(n)) \times \log_2(n)) = O(n \times \log_2(\log_2(n)) \times \log_2(n))$. This gives us the overall time complexity of the `foo()` function: $O(n \times \log_2(\log_2(n)) \times \log_2(n))$.

```

1 void foo(int32_t n) {
2   for (int32_t i = 1; i < n; ++i) {
3     for (int32_t j = 1; j < log2(n); j *= 2) {
4       for (int32_t k = 1; k < n; k *= 2) {
5         std::cout << "Potato!\n";
6       } // for k
7     } // for j
8   } // for i
9 } // foo()

```



Example 4.9 Express the time complexity of the `power_sum()` function using big-O notation, in terms of the input value n .

```

1 int32_t power_sum(int32_t n) {
2   int32_t total = 0;
3   int32_t i = 1;
4   while (i < n) {
5     total += i;
6     i *= 2;
7   } // while
8   return total;
9 } // power_sum()

```

Lines 2, 3, 5, 6, and 8 are all constant time operations. Thus, the time complexity of this function will be determined by the number of times the while loop on line 4 runs. The loop runs as long as i is less than n , and i is doubled with each iteration of the loop. Thus, the number of times the while loop runs is $O(\log_2(n))$. Since the body of the loop runs in $O(1)$ time, the time it takes for the entire while loop to execute is $O(\log_2(n))$ iterations $\times O(1)$ work per iteration $= O(\log_2(n))$.

```

1 int32_t power_sum(int32_t n) {
2   int32_t total = 0;
3   int32_t i = 1;
4   while (i < n) {
5     total += i;
6     i *= 2;
7   } // while
8   return total;
9 } // power_sum()

```



Altogether, the time complexity of the `power_sum()` function is $O(1) + O(1) + O(\log_2(n)) + O(1) = O(\log_2(n))$.

※ 4.4.2 Does the Base of a Logarithm Matter?

When dealing with logarithmic complexity terms, we often ignore the base of the logarithm. It turns out that it does not matter whether an algorithm is $O(\log_2(n))$ or $O(\log_3(n))$; we treat both as part of the same complexity class of $O(\log(n))$. This is because we can always convert a log with one base to a log with another base by multiplying it with a constant, thanks to the *change of base formula* for logarithms:

$$\log_b(n) = \frac{\log_a(n)}{\log_a(b)}$$

To change a logarithmic expression with base a to an identical expression with base b , we simply have to multiply the base- a log with the constant $\frac{1}{\log_a(b)}$. As a result, the base can be extracted out as a coefficient and dropped from our final complexity class!

Example 4.10 Show that $\log_2(n)$ is both $O(\log_2(n))$ and $O(\log_3(n))$.

$f(n)$ is by definition also $O(f(n))$, so $\log_2(n)$ must be $O(\log_2(n))$. To show that $\log_2(n)$ is also $O(\log_3(n))$, we will use the change of base formula:

$$\log_2(n) = \frac{\log_3(n)}{\log_3(2)}$$

In other words, $\log_2(n)$ can be rewritten as $\frac{\log_3(n)}{\log_3(2)}$. This means that

$$\log_2(n) = O(\log_2(n)) = O\left(\frac{\log_3(n)}{\log_3(2)}\right) = O\left(\frac{1}{\log_3(2)} \times \log_3(n)\right)$$

Since we can ignore coefficients when expressing asymptotic time complexity, we can ignore the $\frac{1}{\log_3(2)}$ term, since it is a constant (≈ 1.58):

$$\log_2(n) = O\left(\frac{1}{\log_3(2)} \times \log_3(n)\right) = O(\log_3(n))$$

Thus, we have shown that $\log_2(n)$ is also $O(\log_3(n))$, and that $O(\log_2(n))$ and $O(\log_3(n))$ must be a part of the same complexity class! The same applies for any base. Because of the change of base formula, two logarithms with different bases only differ by a *constant* factor. As shown above, the difference between $\log_3(n)$ and $\log_2(n)$ is a constant factor of approximately 1.58. As a result, if you are in a logarithmic complexity class, it does not matter if the base is 2 or 3 or 999. While a $\log_3(n)$ algorithm does run faster than a $\log_2(n)$ algorithm for large enough input sizes, the speed difference should only differ by a constant amount that *does not grow as the input size grows*. Therefore, the base of the logarithm does not affect the growth of an algorithm, and thus is ignored in big-O notation.

Note that this does **NOT** apply to exponents! The base of an exponent **does** matter. For example, $O(2^n)$ and $O(8^n)$ are not part of the same complexity class. This is because $8^n = 2^n \times 2^{2n}$, and 2^{2n} is not a constant factor!

※ 4.4.3 Logarithmic and Power Identities

The following identities may be useful for working with time complexities that involve logarithms:

Log Identities	Power Identities
1) $\log_a(xy) = \log_a(x) + \log_a(y)$	1) $a^{n+m} = a^n a^m$
2) $\log_a\left(\frac{x}{y}\right) = \log_a(x) - \log_a(y)$	2) $a^{n-m} = \frac{a^n}{a^m}$
3) $\log_a(x^r) = r \log_a(x)$	3) $(a^n)^m = a^{nm}$
4) $\log_a\left(\frac{1}{x}\right) = -\log_a(x)$	4) $a^{-n} = \frac{1}{a^n}$
5) $\log_a(x) = \frac{\log(x)}{\log(a)}$	
6) $\log_a(a) = 1$	
7) $\log_a(1) = 0$	

Example 4.11 An algorithm has a runtime that can be expressed using the following equation. Express the asymptotic time complexity of this algorithm using big-O notation.

$$T(n) = \log(n) + \log(n^{17} \times 3^{n+5})$$

We will use the identities above to simplify the expression. First, using log identity 1, we can simplify $\log(n^{17} \times 3^{n+5})$ to $\log(n^{17}) + \log(3^{n+5})$. This leaves us with

$$T(n) = \log(n) + \log(n^{17}) + \log(3^{n+5})$$

Using log identity 3, we can further simplify $\log(n^{17})$ to $17 \log(n)$. This leaves us with

$$T(n) = \log(n) + 17 \log(n) + \log(3^{n+5})$$

Using power identity 1, we can simplify $\log(3^{n+5})$ to $\log(3^n \times 3^5)$. We can then use log identity 1 again to simplify $\log(3^n \times 3^5)$ further to $\log(3^n) + \log(3^5)$. This leaves us with

$$T(n) = \log(n) + 17 \log(n) + \log(3^n) + \log(3^5)$$

We can use log identity 3 again to simplify $\log(3^n)$ into $n \log(3)$. This leaves us with

$$T(n) = \log(n) + 17 \log(n) + n \log(3) + \log(3^5) = 18 \log(n) + n \log(3) + \log(243)$$

After dropping all coefficients and lower order terms in this equation, we can see that the highest order term here is n (from the $n \log(3)$, but $\log(3)$ is a constant coefficient, so it is dropped). Thus, $T(n) = O(n)$.

Example 4.12 Express the time complexity of the `foo()` function using big-O notation, in terms of m and n , where m and n are positive integers greater than 1.

```

1  int32_t foo(int32_t m, int32_t n) {
2      int32_t c = 0;
3      while (m > n) {
4          m = m / 2;
5          for (int32_t i = 1; i < n; i *= 3) {
6              ++c;
7          } // for i
8      } // while
9      return c;
10 } // foo()
```


Lines 2, 4, 6, and 9 are all constant time operations. We can tell that the inner for loop on line 5 runs a total of $\log_3(n)$ times since i is tripled with each iteration. However, how many times does the outer while loop on line 3 run, in terms of m and n ?

To solve this, first imagine that the while loop condition were $m > 1$ instead of $m > n$. How many times would the while loop run? Since m is halved with each iteration, the outer while loop would run $\log_2(m)$ times.

However, we are stopping the loop once m reaches n , not when it reaches 1. As a result, we do not fully run the loop $\log_2(m)$ times. Instead, we lose $\log_2(n)$ iterations if the loop is stopped when m reaches n instead of when m reaches 1. Let's use an example to visualize this. Suppose m starts off with a value of 256. If we allowed the loop to run as long as $m > 1$, the loop would run a total of $\log_2(256) = 8$ times:

256 128 64 32 16 8 4 2

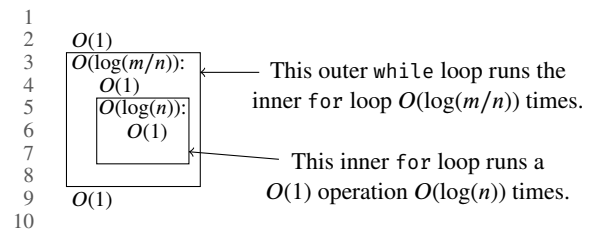
Instead, let's choose the value of n to be 32. How many iterations does the while loop run now? In this case, we only end up running the first three iterations, since the loop now terminates when m reaches 32:

256 128 64 32 16 8 4 2

We lost $\log_2(32) = 5$ iterations of the loop when we set $n = 32$, and only $\log_2(256) - \log_2(32)$ loop iterations were run. In fact, for any m and n , the total number of times the while loop runs is equal to $\log_2(m) - \log_2(n)$, which can be rewritten as $\log_2(m/n)$ using log identities.

```

1  int32_t foo(int32_t m, int32_t n) {
2      int32_t c = 0;
3      while (m > n) {
4          m = m / 2;
5          for (int32_t i = 1; i < n; i *= 3) {
6              ++c;
7          } // for i
8      } // while
9      return c;
10 } // foo()
```



Since the inner loop executes in $O(\log(n))$ time, and the outer loop runs the inner loop $O(\log(m/n))$ times, the overall time complexity of the `foo()` function is $O(\log(m/n) \times \log(n))$.

Remark: A more direct way to solve this problem is to notice that m is halved at each iteration in the outer loop, which means that the value of m at the k^{th} iteration of this loop must be $m \times (1/2)^k$. Since our loop stops once m becomes equivalent to n , the total number of iterations run by the while loop can be expressed as k in the following equation:

$$m \times (1/2)^k = n$$

Solving for the number of iterations k , we get $k = \log_2(m/n)$. Therefore, the outer while loop runs the inner for loop $\log_2(m/n)$ times. Since the complexity of the inner loop is $O(\log(n))$, the overall time complexity of the function is $O(\log(m/n) \times \log(n))$.

4.5 Loop Dependencies

Consider the following function. What is its time complexity? Assume that the input size n is the variable size.

```

1  int32_t foo(int32_t arr[], size_t size) {
2      int32_t count = 0;
3      for (size_t i = size; i > 1; i /= 2) {
4          for (size_t j = 0; j <= i; ++j) {
5              ++count;
6          } // for j
7      } // for i
8      return count;
9  } // foo()
```

This problem may look relatively straightforward upon first glance. You might be tempted to say that the time complexity is $O(n \log(n))$, since the outer loop appears to run $O(\log(n))$ times, and the inner loop appears to run $O(n)$ times. However, this is not accurate! **This is because the terminating condition of the inner loop depends on a value that is changing with the outer loop.** Notice that the value of i used to terminate the inner loop on line 4 depends on the current iteration of the outer for loop on line 3; as a result, the inner loop runs a different number of steps each time. Because of this, the interaction between the two loops cannot be determined if each loop is analyzed individually. To approach the problem, you must consider the entire loop dependency together and count the number of steps that occur with each iteration of the outermost loop in the dependency. This procedure is shown in the following examples.

Remark: For solving loop dependency problems, it may be useful to know the equation for the sum of a geometric series. Given a sequence where the number of terms is n , the first term is a , and the common ratio between terms is r , the sum of the first n terms can be expressed using the following equation:

$$S_n = \frac{a(1-r^n)}{1-r}, \text{ where } r \neq 1$$

Example 4.13 Express the time complexity of the `foo()` function using big-O notation. Let the input size n be the size of the array.

```

1  int32_t foo(int32_t arr[], size_t size) {
2      int32_t count = 0;
3      for (size_t i = size; i > 1; i /= 2) {
4          for (size_t j = 0; j <= i; ++j) {
5              ++count;
6          } // for j
7      } // for i
8      return count;
9  } // foo()

```

There is a loop dependency in this function: the inner `for` loop on line 4 depends on the value of `i` in the outer `for` loop on line 3. Thus, we will analyze both `for` loops together and count the total number of steps that occur with each iteration of the outer loop. On the first iteration of the outer loop, the value of `i` is n , or the input size. Therefore, the inner loop would also run n times on the first iteration of the outer loop. On the second iteration of the outer loop, the value of `i` is halved to $n/2$, so the inner loop will also run $n/2$ times. We can see that the value of `i` is halved with each iteration, which also halves the number of times the inner loop is run. Thus, the runtime of the outer loop can be expressed as:

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \frac{n}{32} + \dots$$

since the inner loop runs n times on the first iteration, $n/2$ times on the second iteration, $n/4$ times on the third iteration, and so on. Since the outer loop halves the value of `i` until it reaches a value of 1, the inner loop must run a total of $\log_2(n)$ times. After factoring out the n from our expression, we get the following:

$$T(n) = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots \right) = n \sum_{k=0}^{\log_2(n)} \left(\frac{1}{2} \right)^k$$

To solve this, we can use the equation for the sum of a geometric series, where the number of terms is $\log_2(n) + 1$, the common ratio is $1/2$, and the first term is 1:

$$T(n) = n \sum_{k=0}^{\log_2(n)} \left(\frac{1}{2} \right)^k \approx n \left[\frac{1 - \left(\frac{1}{2} \right)^{\log_2(n)+1}}{1 - \frac{1}{2}} \right] = n \left[2 \left(1 - \left(\frac{1}{2} \right)^{\log_2(n)+1} \right) \right] = n \left[2 - 2 \left(\left(\frac{1}{2} \right)^{\log_2(n)+1} \right) \right]$$

We can see that the coefficient $2 - 2((1/2)^{\log_2(n)+1})$ is a constant value that is roughly equal to 2, since the term that is subtracted approaches zero as n gets large. Thus, we can conclude that $n(2 - 2((1/2)^{\log_2(n)+1})) \approx 2n = O(n)$, and that the nested loop therefore takes $O(n)$ time.

```

1  int32_t foo(int32_t arr[], size_t size) {
2      int32_t count = 0;
3      for (size_t i = size; i > 1; i /= 2) {
4          for (size_t j = 0; j <= i; ++j) {
5              ++count;
6          } // for j
7      } // for i
8      return count;
9  } // foo()

```

```

1
2  O(1)
3
4
5  O(n)
6
7
8  O(1)
9

```

The number of steps that this nested loop takes can be expressed using the summation $n(1 + 1/2 + 1/4 + 1/8 + 1/16 + \dots) \approx 2n$. Thus, the complexity of this nested loop is $O(n)$.

Since lines 2 and 8 take constant time, the $O(n)$ nested `for` loop on line 3 takes the most time and is thus the dominant term of the entire function. The time complexity of `foo()` is therefore $O(n)$.

Example 4.14 Express the time complexity of the `foo()` function using big-O notation, in terms of the input size n .

```

1  void foo(int32_t n) {
2      for (int32_t i = 0; i < n; ++i) {
3          for (int32_t j = 1; j < n; j *= 3) {
4              for (int32_t k = 0; k < j; ++k) {
5                  std::cout << "Potato!\n";
6              } // for k
7          } // for j
8      } // for i
9  } // foo()

```

Here, we have a loop dependency on lines 3 and 4. The number of times the loop on line 4 runs depends on the value of `j`, which changes in the `for` loop on line 3. Thus, we will need to consider these two loops together.

On the first iteration of the line 3 loop, the line 4 loop runs 1 time. On the second iteration of the line 3 loop, the line 4 loop runs 3 times. On the third iteration of the line 3 loop, the line 4 loop runs 9 times. This continues until the value of `j` reaches n . From this, we can see that the line 3 loop runs the following number of iterations (using the geometric series formula, where $a = 1$, $r = 3$, and $n = \log_3(n) + 1$):

$$1 + 3 + 9 + 27 + \dots + n = \sum_{k=0}^{\log_3(n)} 3^k = \frac{1(1 - 3^{\log_3(n)+1})}{1 - 3} = \frac{1 - 3(3^{\log_3(n)})}{1 - 3} = \frac{1 - 3n}{1 - 3} = \frac{3n - 1}{2} = O(n)$$

Thus, the loop on line 3 runs in $O(n)$ time:

```

1  void foo(int32_t n) {
2      for (int32_t i = 0; i < n; ++i) {
3          for (int32_t j = 1; j < n; j *= 3) {
4              for (int32_t k = 0; k < j; ++k) {
5                  std::cout << "Potato!\n";
6              } // for k
7          } // for j
8      } // for i
9  } // foo()

```

```

1
2  O(n):
3
4
5  O(n)
6
7
8
9

```

The number of steps that this nested loop takes can be expressed using the summation $1 + 3 + 9 + 27 + 81 + \dots + n \approx 1.5n$. Thus, the complexity of this nested loop is $O(n)$.

We can see that the loop on line 2 runs n times, so the loop on line 3 must be executed n times as well. Therefore, the time complexity of this function is $O(n) \times O(n) = O(n^2)$, since the $O(n)$ loop on line 3 is run n times in the loop on line 2.

4.6 Big-O, Big-Θ, and Big-Ω

In the previous few sections, we measured asymptotic time complexities using big-O. In fact, this is the notation that you will likely see in industry. However, by its official definition, the letter "O" on its own may not be specific enough to express certain time complexities. In this section, we will expand upon the definition of big-O notation by introducing two new notations: big-Θ and big-Ω.

Even though we used big-O as our only notation in the previous sections, the O-notation is actually used to denote an **asymptotic upper bound**. The formal definition of big-O is as follows:

A function $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all input sizes $n \geq n_0$.

Big-O can often be used to denote a worst-case time complexity. If an algorithm has a time complexity of $O(g(n))$, it means that the algorithm's runtime growth will be no worse than $g(n)$ for input size n . For example, a $O(n^2)$ algorithm will scale quadratically with respect to input size *at worst*. Big-O notation is similar to a *less than or equal to* relationship. If $T(n) = O(g(n))$, $T(n) \leq g(n)$ as input size n grows without bound.

In addition to big-O, we also have big-Ω, which represents an **asymptotic lower bound**. The formal definition of big-Ω is as follows:

A function $f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all input sizes $n \geq n_0$.

Big-Ω can often be used to denote a best-case time complexity. If an algorithm has a time complexity of $\Omega(g(n))$, it means that the algorithm's runtime growth will be no better than $g(n)$ for input size n . For example, a $\Omega(n^2)$ algorithm will scale quadratically with respect to input size *at best*. Big-Ω notation is similar to a *greater than or equal to* relationship. If $T(n) = \Omega(g(n))$, $T(n) \geq g(n)$ as n grows without bound.

Notice that, while big-O and big-Ω are upper and lower bounds, they are *not always tight* bounds. If someone came to you and asked you how many students were enrolled in EECS 281 this semester, it would not be wrong to say "less than one million" or "more than five." The same thing applies to big-O and big-Ω. Because big-O specifies an upper bound, any algorithm that grows slower than $g(n)$ can be treated as a $O(g(n))$ algorithm. You could assign a constant time algorithm a time complexity of $O(2^n)$, and it would not be wrong because $O(1)$ grows slower than $O(2^n)$! However, much like the "less than one million" response, this does not provide any useful information. The same applies for the "more than five" response for big-Ω. An algorithm that runs in exponential time is technically also $\Omega(1)$, but saying that this algorithm will not run faster than constant time does not provide any meaningful analysis.

To address this issue, we use something called big-Θ to denote an **asymptotic tight bound**. Big-Θ is the tightest bound we can assign to an algorithm. For example, an algorithm with a $O(n^2)$ complexity will not experience anything worse than quadratic growth, and an algorithm with a $\Omega(n^2)$ complexity will not experience anything better than quadratic growth. However, if you are given a $\Theta(n^2)$ algorithm, you can expect its runtime to scale quadratically as the input size gets larger. The formal definition of big-Θ is defined as follows:

A function $f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1 , c_2 and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all input sizes $n \geq n_0$.

Big-Θ is similar to an *equal to* relationship. If $T(n) = \Theta(g(n))$, we can expect $T(n)$ and $g(n)$ to have the same order of growth. **In this course, we will typically use the tightest possible bound to denote the asymptotic complexity of an algorithm.** (In fact, the complexities from the earlier examples could have been more accurately denoted using big-Θ rather than big-O.)

By definition, if a function $T(n)$ is both $O(g(n))$ and $\Omega(g(n))$, it is also $\Theta(g(n))$. Like with the $\leq$, $\geq$, and $=$ operations, if $T(n) \leq g(n)$ and $T(n) \geq g(n)$ are **both** true, then $T(n) = g(n)$ must also be true.

Complexity Rules: The following rules can be used when dealing with asymptotic time complexities.

- **Rules of transitivity:**

1. $f(n) = \Theta(g(n))$ AND $g(n) = \Theta(h(n))$ IMPLY $f(n) = \Theta(h(n))$
 - If $f(n)$ grows at the same rate as $g(n)$, and $g(n)$ grows at the same rate as $h(n)$, then $f(n)$ must grow at the same rate as $h(n)$.
2. $f(n) = O(g(n))$ AND $g(n) = O(h(n))$ IMPLY $f(n) = O(h(n))$
 - If $f(n)$ grows slower than or equal to $g(n)$, and $g(n)$ grows slower than or equal to $h(n)$, then $f(n)$ must grow slower than or equal to $h(n)$.
3. $f(n) = \Omega(g(n))$ AND $g(n) = \Omega(h(n))$ IMPLY $f(n) = \Omega(h(n))$
 - If $f(n)$ grows faster than or equal to $g(n)$, and $g(n)$ grows faster than or equal to $h(n)$, then $f(n)$ must grow faster than or equal to $h(n)$.

- **Rules of reflexivity (i.e., a function is big-O of itself):**

1. $f(n) = O(f(n))$
2. $f(n) = O(f(n))$
3. $f(n) = \Omega(f(n))$

- **Rules of symmetry and transpose symmetry:**

1. $f(n) = \Theta(g(n))$ **if and only if** $g(n) = \Theta(f(n))$
 - If $f(n)$ grows at the same rate as $g(n)$, then $g(n)$ must grow at the same rate as $f(n)$.
2. $f(n) = O(g(n))$ **if and only if** $g(n) = \Omega(f(n))$
 - If $f(n)$ grows slower than or equal to $g(n)$, and $g(n)$ must grow faster than or equal to $f(n)$.

When working with complexities, you may also encounter two other notations: little- o and little- ω . These two notations are similar to their counterparts big- O and big- Ω , except that they are used to describe an upper or lower bound that is *not* tight (i.e., *strictly* less than or greater).

More formally:

- A function $f(n)$ is $o(g(n))$ if there exist positive constants c and n_0 such that $f(n) < cg(n)$ for all input sizes $n \geq n_0$.
- A function $f(n)$ is $\omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) > cg(n)$ for all input sizes $n \geq n_0$.

Example 4.15 Show that $\log(n!) = \Theta(n \log(n))$.

This is an interesting question, as it is not intuitive that taking the log of a factorial term would result in something with the same complexity class as $n \log(n)$. However, with an understanding of log identities, we can prove that this is in fact true. For our proof, we will show that $\log(n!)$ is both $O(n \log(n))$ and $\Omega(n \log(n))$ — if both of these are true, then by definition, $\log(n!)$ must also be $\Theta(n \log(n))$.

First, we will show that $\log(n!) = O(n \log(n))$, or that $\log(n!)$ will never grow at a rate that exceeds a constant multiple of $n \log(n)$. To do this, we can separate the individual terms of the factorial, as follows:

$$\log(n!) = \log(n \times (n-1) \times \dots \times 2 \times 1)$$

Using our log identities, we can rewrite this as:

$$\log(n!) = \log(n) + \log(n-1) + \dots + \log(2) + \log(1)$$

We know that this sum is less than $n \log(n)$, which proves that $\log(n!) = O(n \log(n))$.

$$\log(n) + \log(n-1) + \dots + \log(2) + \log(1) \leq \log(n) + \log(n) + \dots + \log(n) + \log(n) = n \log(n)$$

Now, we will show that $\log(n!) = \Omega(n \log(n))$, or that $\log(n!)$ will always grow at least as fast as a constant multiple of $n \log(n)$. Once again, we will start by using log identities to expand the factorial term:

$$\log(n!) = \log(n) + \log(n-1) + \dots + \log(2) + \log(1)$$

However, this time we will remove the lower half of these terms from this sum. That is, we will remove all terms $\log(k)$ where $k < \frac{n}{2}$ (the terms that are grayed out below are removed):

$$\begin{aligned} \log(n!) &= \log(n) + \log(n-1) + \dots + \log\left(\frac{n}{2} + 1\right) + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{2} - 1\right) + \dots + \log(2) + \log(1) \\ \log(n!) &\geq \log(n) + \log(n-1) + \dots + \log\left(\frac{n}{2} + 1\right) + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{2} - 1\right) + \dots + \log(2) + \log(1) \end{aligned}$$

We know that the other terms must be greater than or equal to $\frac{n}{2} \log\left(\frac{n}{2}\right)$, which is what we get if we replaced all remaining terms with $\log\left(\frac{n}{2}\right)$:

$$\log(n!) \geq \log(n) + \log(n-1) + \dots + \log\left(\frac{n}{2} + 1\right) + \log\left(\frac{n}{2}\right) \geq \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{2}\right) + \dots + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{2}\right) = \frac{n}{2} \log\left(\frac{n}{2}\right)$$

Thus, we have shown that $\log(n!) = \Omega\left(\frac{n}{2} \log\left(\frac{n}{2}\right)\right)$. Now, we will need to show that $\frac{n}{2} \log\left(\frac{n}{2}\right) = \Theta(n \log(n))$, which would prove that $\log(n!)$ is also $\Omega(n \log(n))$. To do so, we can rewrite $\log\left(\frac{n}{2}\right)$ as $\log(n) - \log(2)$ using our log identities. $\frac{n}{2} \log\left(\frac{n}{2}\right)$ can thereby be rewritten as:

$$\frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2} (\log(n) - \log(2)) = \frac{n}{2} \log(n) - \frac{n}{2} \log(2) = \frac{1}{2} (n \log(n)) - \frac{\log(2)}{2} (n)$$

Since coefficients and lower-order terms can be dropped, $\frac{1}{2} (n \log(n)) - \frac{\log(2)}{2} (n) = \Theta(n \log(n))$.

Since $\log(n!) = \Omega\left(\frac{n}{2} \log\left(\frac{n}{2}\right)\right)$ and $\frac{n}{2} \log\left(\frac{n}{2}\right) = \Theta(n \log(n))$, we can conclude that $\log(n!) = \Omega(n \log(n))$.

We have successfully shown that $\log(n!)$ is both $O(n \log(n))$ and $\Omega(n \log(n))$. Therefore, $\log(n!) = \Theta(n \log(n))$.

4.7 Complexity Metrics

There are three primary ways we can express the complexity of an algorithm. Given an algorithm and a fixed input size n , we can either calculate its best-, worst-, or average-case time complexity.

- The **best-case time complexity** is the time complexity of an algorithm in the very best case. This is the lower bound on the running time of an algorithm, and it describes the algorithm's behavior under optimal conditions.
- The **worst-case time complexity** is the time complexity of an algorithm in the very worst case. This is the upper bound on the running time of an algorithm, and it describes the worst rate of runtime growth that could occur over all possible inputs. The worst-case time complexity is a common metric you will see, as it provides valuable insight into how bad an algorithm's runtime may be. However, this metric does not take into account the frequency of the worst-case scenario happening, so it is not always the only metric that is considered. In cases where the worst case occurs very rarely, the worst-case time complexity may not accurately reflect the true performance of an algorithm (this is addressed using a concept known as *amortized complexity*, which is discussed in chapter 12).
- The **average-case time complexity** is the average time complexity of an algorithm over all possible inputs of size n . Much like finding an average in a given set of values, the average-case time complexity can be determined by summing up the resource usage of all possible inputs of size n and dividing this sum by the total number of inputs of size n .

Notice that the input size we feed into an algorithm is fixed for all three complexity metrics. When analyzing the best- and worst-case scenarios, we can only change the contents of the input and not its size. For example, it would be misleading to claim that the best-case scenario for a sorting algorithm occurs when the size of the array is set to 1, since that messes with the size of the input. It would be correct to claim, however, that the best case occurs when the input is already sorted, since that does not mess with the input size.

Example 4.16 You are given an array of n integers, and you want to find the position of a number that you know is in the array. You have an algorithm that conducts a linear search through this array to see if the number you want to find is at each position (i.e., it looks at the first element, then the second, then the third, and so on until it finds the number). What is the best-case, worst-case, and average-case time complexity of this algorithm in terms of n ?

The best-case scenario occurs if the number you want to find is the very first number in the array. The algorithm would check the first element, see that it matches the number you want, and return. This would take constant time! Thus, the best-case time complexity of this algorithm is $\Theta(1)$. The following example illustrates the best-case scenario. Suppose you want to find the number 5 in this array — the linear search would find 5 on its very first try, so it only has to do a constant amount of work.

5	2	3	9	6	...	23	17	8	1
0	1	2	3	4	...	$n-4$	$n-3$	$n-2$	$n-1$

The worst-case scenario occurs if the number you want to find is the very last number in the array (or if it does not exist). This would force the algorithm to check every element in the array. Since there are n elements in total, the worst-case time complexity of this algorithm is $\Theta(n)$. The following example illustrates the worst-case scenario. Suppose you want to find the number 1 in this array (and assuming that 1 only appears once) — the linear search would have to check all n elements before it reaches 1, which takes $\Theta(n)$ time.

5	2	3	9	6	...	23	17	8	1
0	1	2	3	4	...	$n-4$	$n-3$	$n-2$	$n-1$

Now, let's find the average-case time complexity. To do so, we will compute the average time complexity over all possible cases when given an input size of n . For the sake of simplicity, we will assume that all cases are uniformly distributed (i.e., the likelihood of the target element ending up at index 0, index 1, ..., index $n-1$ is equal).

First, we have the case where the element we want to find is the 1st element in the array. The number of elements our algorithm would have to check in this case is 1. Second, we have the case where the element we want to find is the 2nd element in the array. The number of elements our algorithm would have to check in this case is 2.

We can continue this process, keeping track of the number of values our algorithm would need to check if the element we want is the 3rd in the array, the 4th in the array, the 5th in the array, ..., up until the n^{th} in the array. After doing this, we would find that:

- 1 comparison operation is needed in the case where the element we want is the 1st element in the array
- 2 comparison operations are needed in the case where the element we want is the 2nd element in the array
- ...
- n comparison operations are needed in the case where the element we want is the n^{th} element in the array

To calculate the average-case time complexity, we would need find the average number of operations out of all of these possible cases. Using the definition of an average, we get:

$$\text{average number of operations} = \frac{\text{total number of operations over all possible cases}}{\text{number of possible cases}} = \frac{1+2+\dots+n}{n} = \frac{\frac{n(n+1)}{2}}{n} = \frac{n+1}{2} = \Theta(n)$$

After carrying out the division, we find that the average number of operations scales with a complexity of $\Theta(n)$. Since each operation takes constant time, the runtime must also scale with a complexity of $\Theta(n)$. Thus, the average-case time complexity of this algorithm is $\Theta(n)$.

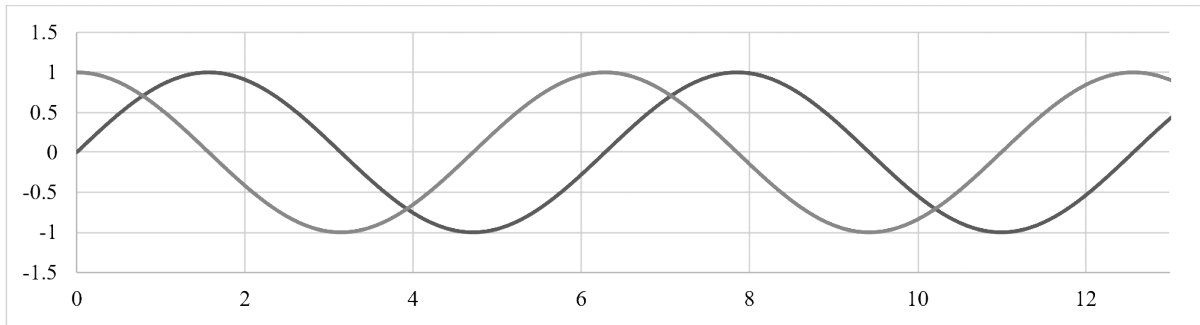
For those with a statistics background, here is a formal proof of the above calculation, presuming that the input is uniformly distributed and that the size of the array is n . If we define X as a random variable that represents the number of comparison operations needed before the target value is found, then the probability that we do x comparisons is $P(X = x) = 1/n$ (since the distribution is uniform). Using the property of expectation, we can determine the expected number of comparison operations as follows:

$$E[X] = \sum_{x=1}^n xP(x) = \sum_{x=1}^n \frac{x}{n} = \frac{1}{n} + \frac{2}{n} + \dots + \frac{n}{n} = \frac{n+1}{2} = \Theta(n)$$

In this class, however, you will likely not have to calculate the average-case time complexity from scratch — instead, you will typically be given the average-case time complexities of important algorithms, without asking you to derive these results on your own. However, you should still understand what an average-case analysis measures and how to calculate it if you ever need to do so.

4.8 Asymptotic Analysis with Limits and L'Hôpital's Rule

It is possible for two functions to have neither a big- O , big- Θ , nor big- Ω relationship. A good example would be the trigonometric functions. For instance, if $f(n) = \sin(n)$ and $g(n) = \cos(n)$, then $f(n)$ is **NOT** $O(g(n))$, $\Theta(g(n))$, nor $\Omega(g(n))$. This is because there is no way to strictly bound one function by another, as shown in the following graph. There is no constant that we can multiply $\cos(n)$ with to have it always grow faster or slower than $\sin(n)$ beyond some input size n_0 . That being said, runtimes that follow a trigonometric pattern are relatively rare.



To show that a function $f(n)$ is $O(g(n))$, the following condition can be used. This condition is *sufficient* but *not necessary*. In other words, the following rule *cannot* be used to prove that $f(n) \neq O(g(n))$.

If $\lim_{n \rightarrow \infty} \left(\frac{f(n)}{g(n)} \right) = d < \infty$, then $f(n) = O(g(n))$. In other words, if the limit of $\frac{f(n)}{g(n)}$ as n grows toward infinity is a constant whose value is less than ∞ , then $f(n)$ is $O(g(n))$.

In cases where n appears in both $f(n)$ and $g(n)$, L'Hôpital's rule may have to be used:

L'Hôpital's Rule: If $f(n)$ and $g(n)$ are differentiable functions and $\lim_{n \rightarrow c} \left(\frac{f(n)}{g(n)} \right) = \frac{0}{0}$ or $\frac{\infty}{\infty}$, then

$$\lim_{n \rightarrow c} \left(\frac{f(n)}{g(n)} \right) = \lim_{n \rightarrow c} \left(\frac{f'(n)}{g'(n)} \right)$$

where $f'(n)$ and $g'(n)$ represent the derivatives of $f(n)$ and $g(n)$, respectively.

Example 4.17 Prove that $\log_2(n)$ is $O(n)$ using the approach introduced in this section.

We can use the condition and L'Hôpital's rule to complete this proof. Let $f(n) = \log_2(n)$ and $g(n) = n$. Using the limit definition, we have:

$$\lim_{n \rightarrow \infty} \left(\frac{\log_2(n)}{n} \right) = \frac{\infty}{\infty}$$

Since we get a result of $\frac{\infty}{\infty}$, we can use L'Hôpital's rule. The derivative of $\log_2(n)$ is $\frac{1}{n \ln(2)}$, and the derivative of n is 1. L'Hôpital's rule gives us the following:

$$\lim_{n \rightarrow \infty} \left(\frac{\log_2(n)}{n} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{d}{dn} [\log_2(n)]}{\frac{d}{dn} [n]} \right) = \lim_{n \rightarrow \infty} \left(\frac{1}{n \ln(2)} \right) = 0 < \infty$$

Since 0 is a constant that is less than ∞ , $\log_2(n)$ must be $O(n)$.

Example 4.18 Prove that $\ln(n)$ is $O\left(\ln\left(\frac{n}{281}\right)\right)$ using the approach introduced in this section.

We can use the same process as above to complete this proof. Here, $f(n) = \ln(n)$ and $g(n) = \ln\left(\frac{n}{281}\right)$. We can simplify $\ln\left(\frac{n}{281}\right)$ to $\ln(n) - \ln(281)$ using logarithm identities. Using the limit definition, we have

$$\lim_{n \rightarrow \infty} \left(\frac{\ln(n)}{\ln(n) - \ln(281)} \right) = \frac{\infty}{\infty}$$

Since we get a result of $\frac{\infty}{\infty}$, we can use L'Hôpital's rule. The derivative of $\ln(n)$ is $\frac{1}{n}$. L'Hôpital's rule gives us the following:

$$\lim_{n \rightarrow \infty} \left(\frac{\ln(n)}{\ln(n) - \ln(281)} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{d}{dn} [\ln(n)]}{\frac{d}{dn} [\ln(n)] - \frac{d}{dn} [\ln(281)]]} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{d}{dn} [\ln(n)]}{\frac{d}{dn} [\ln(n)] - 0} \right) = \lim_{n \rightarrow \infty} \left(\frac{1/n}{1/n} \right) = 1 < \infty$$

Since 1 is a constant that is less than ∞ , $\ln(n)$ must be $O\left(\ln\left(\frac{n}{281}\right)\right)$.

4.9 Space Complexity and Auxiliary Space

In this chapter, we mainly focused on *time complexity*, which deals with how runtime scales with input size. However, runtime is not the only thing that can be expressed using big-O notation. In fact, we can use the notations introduced in this chapter to express the growth of *any computational resource* with respect to input size.

In this section, we will introduce the concept of space complexity (also known as memory complexity), which can be used to describe an algorithm's memory usage. The concept of complexity here is still the same — instead of measuring how runtime scales with input size, we will look at how memory usage scales with input size.

Space complexity is the amount of memory used by an algorithm to solve a problem, with respect to input size.

However, there is a caveat with this definition — since the space complexity looks the total memory usage of an algorithm, it also includes the memory used to store any input values themselves. This is not always something we want! If we do not consider the memory used for the input values themselves in our analysis, we are measuring the *auxiliary space* of the algorithm instead.

Auxiliary space is the temporary space allocated by an algorithm to solve a problem, with respect to input size. Auxiliary space only considers the additional memory used by the algorithm, and it does not include the memory used by the input values themselves.

For this class, we will mostly be dealing with *auxiliary space*. If an algorithm uses $\Theta(1)$ auxiliary space, the amount of additional memory needed to run the algorithm stays constant and does not depend on the input size. If an algorithm uses $\Theta(n)$ auxiliary space, the amount of additional memory needed to run the algorithm grows linearly with the size of the input. Similarly, if an algorithm uses $\Theta(n^2)$ auxiliary space, increasing the input size results in a quadratic increase in the amount of additional memory that is needed.

When dealing with space complexity, it is important to consider all possible memory sources. For example, if a function accepts an argument that is passed in by value (rather than by reference), a copy of that argument is made, which may contribute to auxiliary space. Furthermore, if an algorithm uses recursion, stack frames may be used with each recursive call. If the number of recursive calls depends on input size, the number of stack frames you need may depend on the input size as well! Because stack frames use up memory, an algorithm that does not explicitly allocate memory may still have an auxiliary space complexity worse than $\Theta(1)$.

Example 4.19 What is the auxiliary space complexity of the function `foo()`? The input size n is the size of the input array.

```
1  int32_t foo(int32_t arr[], size_t size) {
2      int32_t* bar = new int32_t[size];
3      int32_t counter = 0;
4      for (size_t i = 0; i < size; ++i) {
5          bar[i] = arr[size - 1 - i];
6      } // for i
7      for (size_t j = 0; j < size; ++j) {
8          counter += (bar[j] + arr[j]);
9      } // for j
10     delete[] bar;
11     return counter;
12 }
```

To determine the auxiliary space of this function, we need to identify where the function allocates memory and express this additional memory usage in terms of the input size. In this case, new memory is allocated on lines 2, 3, 4, and 7. The allocations on lines 3, 4, and 7 instantiate an integer, whose memory usage does not depend on the size of the input array. Thus, we say these lines use $\Theta(1)$ auxiliary space. The allocation on line 2, however, does depend on the size of the input, as the size of the `bar` array is set to the size of the input array. Hence, as input size grows, the amount of memory needed for `bar` also grows. This growth is linear, so the allocation on line 2 uses $\Theta(n)$ auxiliary space.

1	<code>int32_t foo(int32_t arr[], size_t size) {</code>	1	
2	<code>int32_t* bar = new int32_t[size];</code>	2	$\Theta(n)$
3	<code>int32_t counter = 0;</code>	3	$\Theta(1)$
4	<code>for (size_t i = 0; i < size; ++i) {</code>	4	$\Theta(1)$
5	<code>bar[i] = arr[size - 1 - i];</code>	5	
6	<code>} // for i</code>	6	
7	<code>for (size_t j = 0; j < size; ++j) {</code>	7	$\Theta(1)$
8	<code>counter += (bar[j] + arr[j]);</code>	8	
9	<code>} // for j</code>	9	
10	<code>delete[] bar;</code>	10	
11	<code>return counter;</code>	11	
12	<code>} // foo()</code>	12	

Similar to before, we can remove coefficients and lower order terms when expressing space complexity. Because the $\Theta(n)$ memory allocation on line 2 dominates the $\Theta(1)$ allocations elsewhere in the function, the overall auxiliary space used by `foo()` is $\Theta(n)$.

Remark: If the size of `bar` did not depend on `size` (e.g., if the size of `bar` were set to 500 instead of `size` on line 2), then the auxiliary memory used by `bar` would be $500 = \Theta(1)$. If this were the case, the auxiliary space used by the entire `foo()` function would also be $\Theta(1)$, since the additional memory needed to execute the function would not depend on input size (i.e., would not change as the input size grows).

4.10 Profiling and Performance Measurement Tools

※ 4.10.1 Introduction to Profiling

In the previous few sections, we covered the concept of time complexity and how it can be calculated. However, this knowledge is most helpful in allowing us to select the optimal algorithms to use — once we have the algorithms chosen, it becomes our responsibility to implement them correctly. Even if we complete the preliminary analysis perfectly, we may still end up with an inefficient solution if our implementation is faulty.

To make identifying performance issues easier, one important technique to familiarize yourself with is **profiling**, which is a practice that can be used to analyze a program's resource usage experimentally. Using special tools known as *profilers*, you can investigate a program's behavior by directly measuring performance characteristics such as execution time, memory usage, CPU consumption, and I/O activity. A programmer who is adept at profiling can use this powerful strategy to identify where resource bottlenecks occur and quickly pinpoint slow or memory-intensive sections of code. There are a multitude of profilers out there, some of which provide advanced functionality that allow the developer to visualize resource usage with an interactive, user-friendly UI, and you are certainly encouraged to explore these resources while developing your own code. In this section, we will briefly go over two profiling tools that are brought up in the class: `perf` and `massif`.

※ 4.10.2 Measuring Runtime in Linux

Before covering these two profilers, we will deliver a quick aside on how to measure the runtime of a program directly on the Linux command line. To do so, you can use the `time` command, which can be used to measure the execution duration and resource usage of a command or script. For instance, if you wanted to measure the runtime of a program named `exec`, you could run the following:

```
time ./exec
```

After doing this, you will receive output similar to this:

```
real    0m2.059s
user    0m0.345s
sys     0m0.919s
```

Here, **user time** is the time spent by the program doing work. **System time** is the time spent by the operating system on your behalf (such as reading and writing to files — this is not an operation that you implemented, but rather something that is taken care of by the operating system). The **elapsed (real) time** measures real world time; that is, the amount of time that elapsed between the start and end of the program.

The **percentage CPU usage** measures the total time that the CPU was active relative to the real elapsed time, and it can be calculated using the following formula for a specified time interval:

$$\%CPU = \frac{user + system}{elapsed}$$

If a program only uses user and system time, why isn't CPU utilization always 100%? This is because your program does not have access to its own CPU throughout the entire duration of its runtime. Modern operating systems run many processes at once, so each process is assigned a *time quanta*: a specified duration of time that a process can run on a CPU before the CPU is potentially taken away and assigned to another concurrent process. Time quanta are usually on the order of milliseconds, so differences in runtime are often negligible. Nonetheless, percentage CPU utilization decreases as the number of simultaneously running programs increases, as the CPU would need to be shared among a greater number of processes.

※ 4.10.3 Perf

`perf` is a Linux profiling tool that can be used to measure the time spent in different parts of your code by sampling data at different points during a program's execution. To demonstrate, consider the following code:

```
1 void call10() {
2     for (unsigned i = 0; i < 65535 * 1; ++i)
3         std::cout << "Hello world!" << std::endl;
4 } // call10()
5 void call20() {
6     for (unsigned i = 0; i < 65535 * 2; ++i)
7         std::cout << "Hello world!" << std::endl;
8 } // call20()
9 void call30() {
10    for (unsigned i = 0; i < 65535 * 3; ++i)
11        std::cout << "Hello world!" << std::endl;
12 } // call30()
13 void call40() {
14    for (unsigned i = 0; i < 65535 * 4; ++i)
15        std::cout << "Hello world!" << std::endl;
16 } // call40()
17 void function1() {
18     call20();
19     call40();
20 } // function1()
21 void function2() {
22     call10();
23     call30();
24 } // function2()
25 int main() {
26     function1();
27     function2();
28 } // main()
```


We can run this code through `perf` and analyze the output. After doing so, you would get output that looks something like this:

```
perf record --call-graph=dwarf -e cycles:u time ./exec
perf report
```

Terminal					
File	Edit	View	Search	Terminal	Help
Samples:	1K of event		'cpu-clock-u:', Event count (approx.):	338250000	
Children	Self	Command	Shared	Object	Symbol
+ 99.63%	0.00%	demo	demo	[.] main	
+ 90.17%	0.96%	demo	libstdc++.so.6.0.22	[.] std::end<char, std::ch	
+ 89.21%	2.23%	demo	libstdc++.so.6.0.22	[.] std::ostream::put	
+ 86.99%	0.00%	demo	libstdc++.so.6.0.22	[.] std::basic_streambuf<ch	
+ 86.62%	0.00%	demo	libc-2.17.so	[.] _GI_IO_puts (inlined)	
+ 83.96%	0.00%	demo	libc-2.17.so	[.] _IO_10_file_overflow	
+ 83.00%	0.00%	demo	libc-2.17.so	[.] __new_do_write (inlin	
+ 81.15%	0.00%	demo	libc-2.17.so	[.] __new_file_write (inl	
+ 80.04%	80.04%	demo	libc-2.17.so	[.] _GI_libc_write	
+ 80.04%	0.00%	demo	libc-2.17.so	[.] __write_nocancel (inlin	
+ 60.90%	0.00%	demo	demo	[.] function1	
+ 39.91%	0.00%	demo	demo	[.] call40	
+ 38.73%	0.00%	demo	demo	[.] function2	
+ 28.38%	0.00%	demo	demo	[.] call30	
+ 20.92%	0.00%	demo	demo	[.] call20	
+ 10.27%	0.00%	demo	demo	[.] call10	
+ 5.99%	1.48%	demo	libstdc++.so.6.0.22	[.] std::ostream::insert<c	
+ 4.36%	0.00%	demo	libstdc++.so.6.0.22	[.] std::ostream::write	
+ 4.29%	0.00%	demo	libstdc++.so.6.0.22	[.] std::basic_streambuf<	
+ 4.07%	0.00%	demo	libc-2.17.so	[.] _GI_IO_fwrite (inlin	
+ 3.03%	0.37%	demo	libstdc++.so.6.0.22	[.] std::ostream::flush	

TIP: Customize output of perf script with: perf script -F event,ip,sym

At the top of the output, you can see that `perf` collected a total of 1K (1,000) samples while profiling. The bigger this number, the more accurate the results. This is because `perf` collects samples while your program is running by periodically interrupting the execution of its process and recording its current state — this information is essentially used to track the percentage of time that was spent in each function.

Each row of the output contains performance data for a function corresponding with the "Symbol" column. The "Children" column of a function represents the amount of time that was spent in that function along with all other internal functions that it calls.

In the example above, of all 1K samples that were collected, 60.90% were collected while `function1()` was running and 38.73% was collected while `function2()` was running. Furthermore, 39.91% were collected during `call40()`, 28.38% were collected during `call30()`, 20.92% were collected during `call20()`, and 10.27% were collected during `call10()`.

By highlighting a line and pressing enter, you can also expand out each parent function into the child functions that it calls, as shown below:

```

Samples: 1K of event 'cpu-clock:u', Event count (approx.): 33825000
Children  Self Command Shared Object Symbol
- 99.63% 0.00% demo demo [...] main
- - main
  - 60.90% function1
    + 39.91% call40
    + 20.92% call20
  - 38.73% function2
    + 28.38% call30
    + 10.27% call10

```

By using `perf`, you can easily compare the actual percentages measured at runtime with the expected percentages that you expect from the algorithm you implemented. Should the `call40()` function actually take up 40% of the runtime, or is it clearly off? In this case, the 39.91% measurement matches the desired outcome, since the `call40()` function does indeed do 40% of the work. However, if a function returns a percentage that is way larger than expected (e.g., a $\Theta(\log(n))$ algorithm taking up a larger percentage of time than a $\Theta(n^2)$ algorithm), there may be an implementation error in that function.

That being said, you should be careful when analyzing perf results. A larger percentage is not always a bad thing if the function that is taking a long time is expected to take a long time. Furthermore, it is important to check the number of samples that are taken. If the sample size is small, the results you get may be misleading. To increase the number of samples collected, make sure to run the program with a larger input size.

※ 4.10.4 *Massif*

`massif` is a heap profiling tool provided by Valgrind (an instrumentation framework for Linux that is primarily used to detect memory issues, which will be covered in greater detail in chapter 6), and it can be used to measure a program’s memory usage over time. This tool tracks heap allocations and reports when and where memory consumption peaks, making it easier for the developer to identify and fix potential memory issues. As an example, we will consider the following code:

```

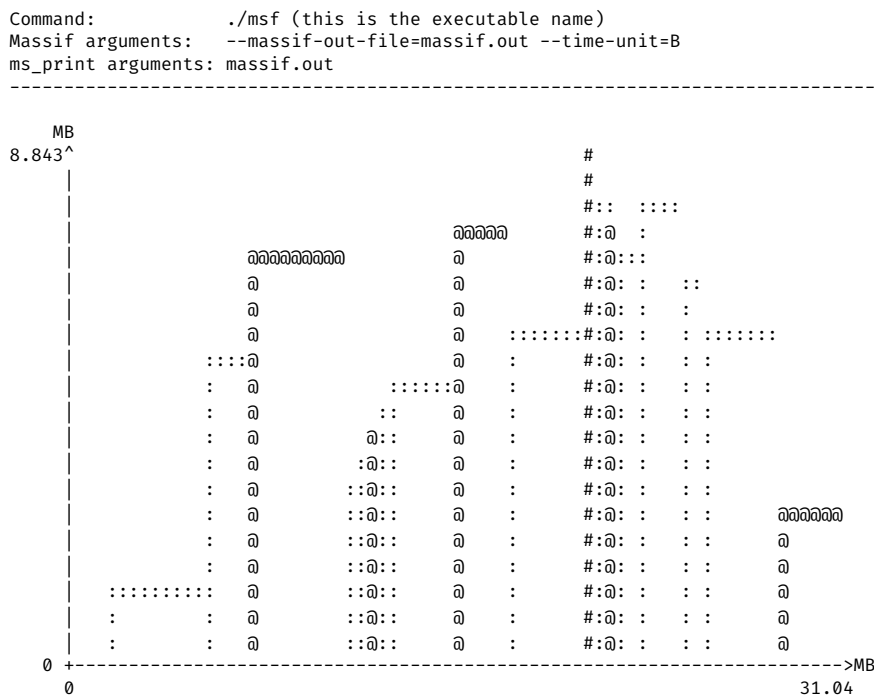
1  int main() {
2      int32_t **a = new int32_t*[200000];
3      int32_t *b = new int32_t[1000000];
4      int32_t *c = new int32_t[500000];
5      delete[] b;
6      for (int32_t i = 0; i < 16; ++i) {
7          a[i] = new int32_t[25000];
8      } // for i
9      int32_t *d = new int32_t[750000];
10     delete[] c;
11     int32_t *e = new int32_t[750000];
12     for (int32_t i = 0; i < 16; ++i) {
13         delete[] a[i];
14     } // for i
15     int32_t *f = new int32_t[250000];
16     delete[] a;
17     delete[] f;
18     delete[] d;
19     delete[] e;
20 } // main()

```

This code will be compiled using the `-g3` flag into an executable named `msf`. The following will be run to generate a massif output file named `massif.out` (make sure to include any flags or input files needed with the executable):

```
valgrind --tool=massif --massif-out-file=massif.out --time-unit=B ./msf
```

After this, we will run the command `"ms_print massif.out"` to get the following output (snapshots 16-49 truncated but will be shown later):



n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
0	0	0	0	0	0
1	72,712	72,712	72,704	8	0
2	1,672,720	1,672,720	1,672,704	16	0
3	5,672,728	5,672,728	5,672,704	24	0
4	7,672,736	7,672,736	7,672,704	32	0
5	7,672,736	7,672,736	7,672,704	32	0
100.00% (7,672,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.					
->52.13% (4,000,000B) 0x4005E5: main (massif.cpp:3)					
->26.07% (2,000,000B) 0x4005F3: main (massif.cpp:4)					
->20.85% (1,600,000B) 0x4005D7: main (massif.cpp:2)					
->00.95% (72,704B) in 1+ places, all below ms_print's threshold (01.00%)					

n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
6	11,672,744	3,672,728	3,672,704	24	0
7	11,772,752	3,772,736	3,772,704	32	0
8	11,872,760	3,872,744	3,872,704	40	0
9	11,972,768	3,972,752	3,972,704	48	0
10	12,072,776	4,072,760	4,072,704	56	0
11	12,172,784	4,172,768	4,172,704	64	0
12	12,272,792	4,272,776	4,272,704	72	0
13	12,372,800	4,372,784	4,372,704	80	0
14	12,472,808	4,472,792	4,472,704	88	0
15	12,572,816	4,572,800	4,572,704	96	0
100.00% (4,572,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.					
->43.74% (2,000,000B) 0x4005F3: main (massif.cpp:4)					
->34.99% (1,600,000B) 0x4005D7: main (massif.cpp:2)					
->19.68% (900,000B) 0x400636: main (massif.cpp:7)					
->01.59% (72,704B) 0x4EB7D4E: _GLOBAL__sub_I_eh_alloc.cc (eh_alloc.cc:123)					
->01.59% (72,704B) 0x400F8F1: _dl_init (in /usr/lib64/ld-2.17.so)					
->01.59% (72,704B) 0x4001158: ??? (in /usr/lib64/ld-2.17.so)					
->00.00% (0B) in 1+ places, all below ms_print's threshold (01.00%)					

Each vertical bar of the graph represents a memory snapshot that was taken during the program's execution. In this output, a total of 50 snapshots were taken (numbered 0-49). Most snapshots only include basic information about total memory usage, but several detailed snapshots provide additional information (in this case, snapshots 5, 15, 24, 27, 37, and 47). To demonstrate, we will look at detailed snapshot 5:

```
-----
n      time(B)      total(B)  useful-heap(B)  extra-heap(B)  stacks(B)
-----
0          0          0          0          0          0
1       72,712       72,712       72,704          8          0
2      1,672,720      1,672,720      1,672,704         16          0
3      5,672,728      5,672,728      5,672,704         24          0
4      7,672,736      7,672,736      7,672,704         32          0
5      7,672,736      7,672,736      7,672,704         32          0
100.00% (7,672,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
->52.13% (4,000,000B) 0x4005E5: main (massif.cpp:3)
|
->26.07% (2,000,000B) 0x4005F3: main (massif.cpp:4)
|
->20.85% (1,600,000B) 0x4005D7: main (massif.cpp:2)
|
->00.95% (72,704B) in 1+ places, all below ms_print's threshold (01.00%)
```

When snapshot 5 was taken, the program was using 7,672,736 bytes of memory in total (the program itself requested 7,672,704 bytes but was given 32 extra heap bytes in excess of what it asked for to handle additional metadata and overhead). The remaining lines under snapshot 5 indicate the memory usage contribution of different objects in the code, listed in sorted order.

- Line 3 allocated 4,000,000 bytes (1,000,000 integers of 4 bytes each), which contributes to 52.13% of total memory usage.
- Line 4 allocated 2,000,000 bytes (500,000 integers of 4 bytes each), which contributes to 26.07% of total memory usage.
- Line 2 allocated 1,600,000 bytes (200,000 pointers of 8 bytes each), which contributes to 20.85% of total memory usage.

Let's continue to the next detailed snapshot, snapshot 15:

```
-----
n      time(B)      total(B)  useful-heap(B)  extra-heap(B)  stacks(B)
-----
6      11,672,744      3,672,728      3,672,704         24          0
7      11,772,752      3,772,736      3,772,704         32          0
8      11,872,760      3,872,744      3,872,704         40          0
9      11,972,768      3,972,752      3,972,704         48          0
10     12,072,776      4,072,760      4,072,704         56          0
11     12,172,784      4,172,768      4,172,704         64          0
12     12,272,792      4,272,776      4,272,704         72          0
13     12,372,800      4,372,784      4,372,704         80          0
14     12,472,808      4,472,792      4,472,704         88          0
15     12,572,816      4,572,800      4,572,704         96          0
100.00% (4,572,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
->43.74% (2,000,000B) 0x4005F3: main (massif.cpp:4)
|
->34.99% (1,600,000B) 0x4005D7: main (massif.cpp:2)
|
->19.68% (900,000B) 0x400636: main (massif.cpp:7)
|
->01.59% (72,704B) 0x4EB7D4E: _GLOBAL__sub_I_eh_alloc.cc (eh_alloc.cc:123)
|
->01.59% (72,704B) 0x400F8F1: _dl_init (in /usr/lib64/ld-2.17.so)
|
->01.59% (72,704B) 0x4001158: ??? (in /usr/lib64/ld-2.17.so)
|
->00.00% (0B) in 1+ places, all below ms_print's threshold (01.00%)
```

At this point, 4,572,800 bytes are currently in usage. 43.74% of the memory currently being used was created on line 4, 34.99% was created on line 2, and 19.68% was created on line 7. Notice that the memory allocated by line 3 no longer contributes to the memory usage since it was deallocated on line 5. From the output, we can see that the deallocation happened between snapshots 5 and 6 due to the drop in total memory usage by 4,000,000 bytes. (Note that the `time` column is measured by total number of allocation and deallocation events processed — if you want to change this unit to milliseconds of real time, you can run `massif` with the flag `--time-unit=ms` instead).

From the snapshots, you can actually see the memory being profiled as the `for` loop on line 6 runs. Every snapshot after snapshot 6 was taken after one additional iteration of the `for` loop, which allocates 100,000 bytes of new memory (via 25,000 integers each of size 4 bytes). When snapshot 15 was taken, the `for` loop had run 9 times, so the total memory usage allocated by the loop at this point is 900,000 bytes. The remaining memory allocated by the `for` loop is completed 7 additional iterations later, as shown by snapshots 16-22:

```
-----
n      time(B)      total(B)  useful-heap(B)  extra-heap(B)  stacks(B)
-----
16     12,672,824      4,672,808      4,672,704         104         0
17     12,772,832      4,772,816      4,772,704         112         0
18     12,872,840      4,872,824      4,872,704         120         0
19     12,972,848      4,972,832      4,972,704         128         0
20     13,072,856      5,072,840      5,072,704         136         0
21     13,172,864      5,172,848      5,172,704         144         0
22     13,272,872      5,272,856      5,272,704         152         0
```

Snapshots 23 and 24 correspond the allocation on line 9 and deallocation on line 10. At this point, the array `d` contributes the most toward total memory consumption, with 36.26%.

... continued from previous page ...

```

22 13,272,872      5,272,856      5,272,704      152      0
23 16,272,880      8,272,864      8,272,704      160      0
24 16,272,880      8,272,864      8,272,704      160      0
100.00% (8,272,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
->36.26% (3,000,000B) 0x400649: main (massif.cpp:9)
|
->24.18% (2,000,000B) 0x4005F3: main (massif.cpp:4)
|
->19.34% (1,600,000B) 0x4005D7: main (massif.cpp:2)
|
->19.34% (1,600,000B) 0x400636: main (massif.cpp:7)
|
->00.88% (72,704B) in 1+ places, all below ms_print's threshold (01.00%)

```

Now, we reach our peak allocation during snapshot 27. This occurs right before line 12 of the code:

```

-----
n      time(B)      total(B)      useful-heap(B) extra-heap(B)      stacks(B)
-----
25 18,272,888      6,272,856      6,272,704      152      0
26 21,272,896      9,272,864      9,272,704      160      0
27 21,272,896      9,272,864      9,272,704      160      0
100.00% (9,272,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
->32.35% (3,000,000B) 0x400649: main (massif.cpp:9)
|
->32.35% (3,000,000B) 0x40066A: main (massif.cpp:11)
|
->17.25% (1,600,000B) 0x4005D7: main (massif.cpp:2)
|
->17.25% (1,600,000B) 0x400636: main (massif.cpp:7)
|
->00.78% (72,704B) in 1+ places, all below ms_print's threshold (01.00%)

```

Since assignments may be graded on peak memory usage, this is likely the most important snapshot if you are investigating memory consumption issues. At this point, array `d` allocated on line 9 and array `e` allocated on line 11 take up the most memory, with 32.35% contributed by each. If your program is consistently over memory, this is the section that you want to investigate — is this memory usage expected by this part of the code? A good tip would be to look for unnecessary copies of large objects, which could be allocating additional memory without your intention.

Example 4.20 The next few snapshots of the output are shown below. At which point of the function's execution was snapshot 37 taken?

```

-----
n      time(B)      total(B)      useful-heap(B) extra-heap(B)      stacks(B)
-----
28 21,372,904      9,172,856      9,172,704      152      0
29 21,472,912      9,072,848      9,072,704      144      0
30 21,572,920      8,972,840      8,972,704      136      0
31 21,672,928      8,872,832      8,872,704      128      0
32 21,772,936      8,772,824      8,772,704      120      0
33 21,872,944      8,672,816      8,672,704      112      0
34 21,972,952      8,572,808      8,572,704      104      0
35 22,072,960      8,472,800      8,472,704      96      0
36 22,172,968      8,372,792      8,372,704      88      0
37 22,272,976      8,272,784      8,272,704      80      0
100.00% (8,272,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
->36.26% (3,000,000B) 0x400649: main (massif.cpp:9)
|
->36.26% (3,000,000B) 0x40066A: main (massif.cpp:11)
|
->19.34% (1,600,000B) 0x4005D7: main (massif.cpp:2)
|
->07.25% (600,000B) 0x400636: main (massif.cpp:7)
|
->00.88% (72,704B) in 1+ places, all below ms_print's threshold (01.00%)

```

We can see that the total memory usage is decreasing by 100,000 bytes per snapshot, indicating that we are in the loop on line 12 that is deallocating memory from array `a`. Notice that the memory usage from array `a` was 1,600,000 bytes at peak, and is now 600,000 bytes after snapshot 37. Since each iteration of the loop deallocates 100,000 bytes (in the form of 25,000 integers of size 4 bytes), snapshot 37 must have been taken after 10 iterations of the loop on line 12.

Example 4.21 The remaining snapshots of the output are shown below. At which point of the function's execution was snapshot 47 taken?

n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
38	22,372,984	8,172,776	8,172,704	72	0
39	22,472,992	8,072,768	8,072,704	64	0
40	22,573,000	7,972,760	7,972,704	56	0
41	22,673,008	7,872,752	7,872,704	48	0
42	22,773,016	7,772,744	7,772,704	40	0
43	22,873,024	7,672,736	7,672,704	32	0
44	23,873,032	8,672,744	8,672,704	40	0
45	25,473,040	7,072,736	7,072,704	32	0
46	26,473,048	6,072,728	6,072,704	24	0
47	29,473,056	3,072,720	3,072,704	16	0

100.00% (3,072,704B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
 ->97.63% (3,000,000B) 0x40066A: main (massif.cpp:11)
 |
 ->02.37% (72,704B) 0x4EB7D4E: _GLOBAL__sub_I_eh_alloc.cc (eh_alloc.cc:123)
 | ->02.37% (72,704B) 0x400F8F1: _dl_init (in /usr/lib64/ld-2.17.so)
 | ->02.37% (72,704B) 0x4001158: ??? (in /usr/lib64/ld-2.17.so)
 |
 ->00.00% (0B) in 1+ places, all below ms_print's threshold (01.00%)

n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
48	32,473,064	72,712	72,704	8	0
49	32,545,776	0	0	0	0

From the information contained in detailed snapshot 47, we can see that only 3,000,000 bytes are in usage from the memory allocated in `main()`, belonging to array `e` initialized on line 11. Thus, snapshot 47 was taken after `d` was deallocated on line 18. We can also see from the last two snapshots that `e` is deallocated after snapshot 48, and all other memory is cleaned up before the program ends after snapshot 49.

Example 4.22 Consider the following output, which was obtained after running `massif` on another program. What is the value of `X`?

Number of snapshots: 52
 Detailed snapshots: [4, 7, 12, 17, 22, 25 (peak), 30, 35, 40, 45, 51]

n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
46	83,960	75,896	75,232	664	0
47	84,152	75,704	75,200	504	0
48	84,344	75,512	75,168	344	0
49	84,536	75,320	75,136	184	0
50	84,728	75,128	75,104	24	0
51	157,440	2,416	[-X-]	16	0

The total memory usage is the sum of the useful-heap (the exact number of bytes your program explicitly requested via memory allocation) and the extra-heap (additional bytes allocated as overhead associated with memory management). Since the total memory usage in snapshot 51 is 2,416 bytes, and 16 bytes are part of the extra-heap allocation, the useful-heap allocation must be $2,416 - 16 = 2,400$ bytes.

Practice Quiz

Disclaimer: These practice questions are not official and may not have been vetted for course material. You may use these for practice, but you should prioritize official resources from current staff for a more accurate depiction of what you need to know for assignments and exams.

1. Which of the following statements provides an explanation for why it may be better to measure a program's relative efficiency by analyzing how its runtime scales by input size rather than its pure runtime?
 - If you compare two algorithms based on pure runtime, the superior algorithm may end up being measured as less efficient if more processes were running in the background when it was run.
 - For input sizes that are large enough, the rate of growth of a program's runtime with respect to input size is independent of most external factors, making it a consistent benchmark for comparing the growth of different algorithms.
 - Runtime measured with respect to input size can be expressed mathematically using big-O notation, which makes it easier to categorize and compare different algorithms with different orders of growth.
 - I only
 - II only
 - I and II only
 - II and III only
 - I, II, and III

2. Consider the following four statements regarding algorithm complexities:

- I. An algorithm with a $\Theta(n^2)$ time complexity will always run faster than an algorithm with a $\Theta(n \log(n))$ time complexity.
- II. An algorithm with a $\Theta(n \log(n))$ time complexity will always run faster than an algorithm with a $\Theta(n^2)$ time complexity.
- III. An algorithm with a $\Theta(n^2)$ time complexity will always run faster than an algorithm with a $\Theta(n!)$ time complexity.
- IV. An algorithm with a $\Theta(n!)$ time complexity will always run faster than an algorithm with a $\Theta(n^2)$ time complexity.

How many of these statements is/are **TRUE**?

- A) 0
- B) 1
- C) 2
- D) 3
- E) 4

3. Given a function f , which of the following best defines $O(f)$ using big-O notation?

- A) $O(f)$ is the set of functions that grow at least as slowly as f
- B) $O(f)$ is the set of functions that grow at least as quickly as f
- C) $O(f)$ is the set of functions that grow exactly as quickly as f
- D) $O(f)$ is the set of functions that do not grow more slowly than f
- E) $O(f)$ is the set of functions that do not grow more quickly than f

4. You are measuring the asymptotic time complexity of a function *foo* by counting steps. Suppose you determine that the total number of steps taken by *foo* with respect to input size n can be expressed as:

$$T(n) = 53 \log(n) + 79n + 12$$

What of the following statements is **TRUE**?

- A) *foo* has a time complexity of $\Theta(53 \log(n))$
- B) *foo* has a time complexity of $\Theta(\log(n))$
- C) *foo* has a time complexity of $\Theta(n)$
- D) *foo* has a time complexity of $\Theta(n \log(n))$
- E) More than one of the above

5. You are given five different algorithms, each with a different time complexity:

- I. Algorithm A has a time complexity of $\Theta(n!)$
- II. Algorithm B has a time complexity of $\Theta(2^n)$
- III. Algorithm C has a time complexity of $\Theta(n^3)$
- IV. Algorithm D has a time complexity of $\Theta(n^3 \log(n))$
- V. Algorithm E has a time complexity of $\Theta(n^n)$

Which of the following correctly ranks these algorithms in order of increasing time complexity?

- A) D, C, B, A, E
- B) B, C, D, A, E
- C) B, D, C, A, E
- D) C, D, B, A, E
- E) C, D, B, E, A

6. You have n billiard balls, all of which have the same weight except for one that is heavier than all the others. Which of the following is a $\Theta(\log(n))$ solution for finding the billiard ball with a heavier weight using only a single balance?



- A) Randomly pick two balls at a time and place them on each side of the balance until you find an imbalance
- B) Place one ball on one side of the balance, and compare its weight with each of the other balls using the other side of the balance
- C) Split the balls into two halves, place one half on one side of the balance and the other half on the other, and check to see which half is heavier; continue this process with the balls on the heavier side until the target ball is found
- D) Split the balls into three groups, place one third on one side and another third on the other, and check which group the heavier ball is in (if the two groups on the scale have the same weight, then the heavier ball must be in the group not on the scale); continue this process with the group that includes the heavier ball until the target ball is found
- E) More than one of the above

7. You just completed a project after a long day of work, and good news: your code appears to run correctly with the provided test files! However, when you plot out runtimes with your own test files, you notice that there exists a $\Theta(n)$ relationship even though you had implemented a $\Theta(\log(n))$ algorithm. Which of the following could be a reason for this discrepancy?

- A) There were too many programs running in parallel with your program
- B) $\Theta(n)$ algorithms can also have a time complexity of $\Theta(\log(n))$
- C) The tests that you used were too large for the actual relationship to be revealed
- D) The tests that you used were too small for the actual relationship to be revealed
- E) None of the above

8. Your friend is working on a project, which they implemented using a $\Theta(n^2)$ algorithm. However, when they ran a test file on their program, they noticed that the program instead ran in $\Theta(n)$ time. All else equal, which of the following is **NOT** a possible reason for this discrepancy?
- A) Your friend accidentally ran the test case with a different algorithm
 - B) Your friend incorrectly analyzed the time complexity of their algorithm
 - C) Your friend's test inputs exposed worst-case behavior
 - D) Your friend's test case input size was too small
 - E) None of the above

9. Suppose you have three functions f , g , and h . Consider the following statements:
- I. f must either be $O(g)$, $\Theta(g)$, or $\Omega(g)$.
 - II. If $f + g$ is $O(h)$, then either f or g must also be $O(h)$.
 - III. If $f \times g$ is $O(h)$, then either f or g must also be $O(h)$.
 - IV. There exists no f such that f is $\Omega(f \times g)$, where $f \times g$ is the product of two functions f and g .

How many of these statements is/are **TRUE**?

- A) 0
 - B) 1
 - C) 2
 - D) 3
 - E) 4
10. Which of the following statements is **FALSE**?
- A) $\log(n^3 9^n)$ is $\Theta(9^n)$
 - B) $\log(n^5 4^{3n})$ is $\Theta(n)$
 - C) $\log(n!)$ is $\Theta(n \log(n))$
 - D) $\log_3(n)$ is $\Theta(\log_2(n))$
 - E) $\log(n^2)$ is $\Theta(\log(n))$
11. Which of the following strategies can be used to improve the worst-case *asymptotic* time complexity of an algorithm?
- A) Reducing the size of the input that is passed into the algorithm
 - B) Increasing the size of the input that is passed into the algorithm
 - C) Running the algorithm on a faster machine
 - D) Compiling the algorithm using the `-O3` compiler flag
 - E) None of the above

12. Consider the following function implementation:

```

1  int32_t foo(int32_t n) {
2      int32_t a = helper_a(n);
3      int32_t b = helper_b(n);
4      int32_t c = helper_c(n);
5      return a + b + c;
6  } // foo()
```

For an input size of n , the `helper_a()` function has an asymptotic time complexity of $\Theta(n^2)$, the `helper_b()` function has an asymptotic time complexity of $\Theta(n)$, and the `helper_c()` function has an asymptotic time complexity of $\Theta(n \log(n))$. Which of the following modifications would *change* the asymptotic time complexity class of the function `foo()`?

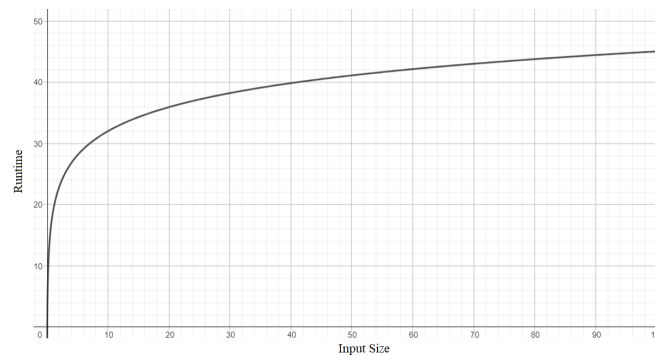
- A) Improving the asymptotic time complexity of `helper_a()` from $\Theta(n^2)$ to $\Theta(n \log(n))$
 - B) Improving the asymptotic time complexity of `helper_a()` from $\Theta(n^2)$ to $\Theta(0.5n^2)$
 - C) Improving the asymptotic time complexity of `helper_b()` from $\Theta(n)$ to $\Theta(1)$
 - D) Improving the asymptotic time complexity of `helper_c()` from $\Theta(n \log(n))$ to $\Theta(\log(n))$
 - E) More than one of the above
13. You are given three algorithms A , B , and C , each with runtimes that can be modeled using the functions $A(n)$, $B(n)$, $C(n)$ for input size n . You know that $A(n) = O(B(n))$ and $A(n) = \Omega(C(n))$. Which of the following statements **CANNOT** be true?
- A) $B(n) = \Theta(C(n))$
 - B) $C(n) = \Omega(A(n))$
 - C) $B(n) = \Omega(C(n))$
 - D) $B(n) = O(C(n))$
 - E) None of the above (i.e., all of the above statements can be true)
14. You are given three strictly increasing functions $A(n)$, $B(n)$, and $C(n)$ for input size n . Suppose you know that $A(n)$ is $O(B(n))$ and $A(n)$ is $\Omega(C(n))$. Which of the following statements must also be **TRUE**?
- A) $B(n)$ is $O(C(n))$
 - B) $B(n)$ is $\Omega(C(n))$
 - C) $B(n)$ is $\Theta(C(n))$
 - D) $B(n)$ is $\Theta(A(n))$
 - E) None of the above

15. You have four algorithms that accomplish the same task, and you want to know which algorithm to use. Memory is not an issue, so you want to make your decision based on time efficiency alone. To make your decision, you run each algorithm once on the same machine at the exact same time. The same input size is used for all the algorithms. You obtain the following results from this experiment:

Algorithm	Runtime (ms)
A	0.011
B	0.015
C	0.006
D	0.019

Using just the information collected from this experiment, which algorithm should you select?

- A) Algorithm A
 B) Algorithm B
 C) Algorithm C
 D) Algorithm D
 E) The algorithm you should select should not be determined using the results of this experiment
16. You are given a function $\text{foo}()$, and you graph its runtime with respect to different input sizes. After running multiple experiments with differing inputs, you end with the following relationship between runtime and input size:



What is the asymptotic time complexity of $\text{foo}()$ with respect to input size n ?

- A) $\Theta(\log(n))$
 B) $\Theta(n)$
 C) $\Theta(n \log(n))$
 D) $\Theta(n \log(\log(n)))$
 E) $\Theta(n^2 \log(n))$
17. Which of the following correctly orders these complexity classes in order from slowest to fastest growth?
- I. $\Theta(\sqrt{\log(n)})$
 II. $\Theta(\log(n))$
 III. $\Theta(\log^2(n))$
 IV. $\Theta(\log(\log(n)))$
- A) I, IV, II, III
 B) I, IV, III, II
 C) I, II, IV, III
 D) IV, I, II, III
 E) IV, I, III, II
18. Suppose you are given a function $f(n)$ that you know is $\Theta(n^2 \log(n))$. Which of the following statements must also be **TRUE** about $f(n)$?
- A) $f(n)$ is $\Theta(n^2)$
 B) $f(n)$ is $\Theta(n^2 + n)$
 C) $f(n)$ is $\Omega(n^2 + \log(n))$
 D) $f(n)$ is $O(n^2)$
 E) More than one of the above
19. Which one of the following statements is **TRUE**?
- A) $\log(n + m)$ is $O(\log(n) + \log(m))$
 B) $n!$ is $O((n - 1)!)$
 C) 2^{n+m} is $O(2^n + 2^m)$
 D) $\sin(n)$ is $O(\cos(n))$
 E) More than one of the above

20. For which of the following pairs of $f(n)$ and $g(n)$ is $f(n) = \Theta(g(n))$? Note: e is a mathematical constant with approximate value 2.71828...
- I. $f(n) = 2^{2n}$
 $g(n) = 2^n$
 - II. $f(n) = 2^{n \log_2(n)}$
 $g(n) = n^n$
 - III. $f(n) = 2e^{n/2}$
 $g(n) = e^n$
- A) I only
 - B) II only
 - C) III only
 - D) I and III only
 - E) II and III only
21. If $f(n)$ and $g(n)$ are real-valued functions such that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, which of the following conclusions are valid?
- A) $f(n)$ is $O(g(n))$
 - B) $f(n)$ is $\Theta(g(n))$
 - C) $f(n)$ is $\Omega(g(n))$
 - D) More than one of the above
 - E) None of the above conclusions can be made from the given information
22. You are given an algorithm whose runtime can be expressed in the form $f(n) = C^n$ for input size n and some unknown constant C . From analysis, you discover that it takes T time for this algorithm to solve a problem with input size S , and $9T$ time for this algorithm to solve a problem with input size $S + 6$. Knowing this, what is the value of the constant C ?
- A) $\sqrt[3]{3}$
 - B) $\sqrt{3}$
 - C) $3/2$
 - D) 2
 - E) 3
23. You are given an algorithm whose runtime can be expressed in the form $f(n) = \log_2(n)$ for input size n . If it takes T time to solve a problem with input size S , a problem of what input size can be solved in time $T + 1$?
- A) $\log_2(S)$
 - B) $S + \log_2(S)$
 - C) $2S$
 - D) $2\log_2(S)$
 - E) $2S + 2\log_2(S)$
24. You are given an algorithm whose runtime can be expressed in the form $f(n) = n^2$ for input size n . If it takes T time to solve a problem with input size S , a problem of what input size can be solved in time $8T$?
- A) $2S$
 - B) $(2\sqrt{2})S$
 - C) $3S$
 - D) $4S$
 - E) $(4\sqrt{2})S$
25. Which of the following correctly lists out these complexity classes in order from slowest to fastest growth?
- A) $\Theta(n^{281})$, $\Theta(n^{280} \log(n!))$, $\Theta(281^n)$
 - B) $\Theta(n^{281})$, $\Theta(281^n)$, $\Theta(n^{280} \log(n!))$
 - C) $\Theta(281^n)$, $\Theta(n^{281})$, $\Theta(n^{280} \log(n!))$
 - D) $\Theta(281^n)$, $\Theta(n^{280} \log(n!))$, $\Theta(n^{281})$
 - E) $\Theta(n^{280} \log(n!))$, $\Theta(n^{281})$, $\Theta(281^n)$
26. Consider the two functions $f(n) = n \sin(n) + n$ and $g(n) = n$. While trying to determine the big-O relationship between these two functions, your friend realizes that:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{n \sin(n) + n}{n} = \lim_{n \rightarrow \infty} \frac{\sin(n) + 1}{1}$$

From this, your friend concludes that $f(n) \neq O(g(n))$ because $\lim_{n \rightarrow \infty} \frac{\sin(n) + 1}{1}$ does not exist. Is your friend correct? If not, why not?

- A) Yes, your friend is correct
- B) No, your friend is incorrect because they computed the limit incorrectly
- C) No, your friend is incorrect because they forgot to apply L'Hôpital's rule before drawing a conclusion
- D) No, your friend is incorrect because a limit that does not exist implies that $f(n)$ is $O(g(n))$
- E) No, your friend is incorrect because the limit condition is sufficient but not necessary for big-O

27. Consider the following function implementation:

```

1  void foo(int32_t m, int32_t n) {
2      for (int32_t i = 0; i < m; ++i) {
3          std::cout << "EECS 281\n";
4      } // for i
5
6      for (int32_t j = 0; j < n; ++j) {
7          std::cout << "EECS 281\n";
8      } // for j
9  } // foo()

```

What is the time complexity of this function in terms of m and n ?

- A) $\Theta(m)$
- B) $\Theta(n)$
- C) $\Theta(m+n)$
- D) $\Theta(mn)$
- E) None of the above

28. Consider the following function implementation:

```

1  void foo(int32_t m, int32_t n) {
2      for (int32_t i = 0; i < m; ++i) {
3          for (int32_t j = 0; j < n; ++j) {
4              std::cout << "EECS 281\n";
5          } // for j
6      } // for i
7  } // foo()

```

What is the time complexity of this function in terms of m and n ?

- A) $\Theta(m)$
- B) $\Theta(n)$
- C) $\Theta(m+n)$
- D) $\Theta(mn)$
- E) None of the above

29. Consider the following function implementation:

```

1  void foo(int32_t m, int32_t n, int32_t p) {
2      for (int32_t k = 0; k < p; ++k) {
3          for (int32_t i = 0; i < m; ++i) {
4              std::cout << "EECS 281\n";
5          } // for i
6
7          for (int32_t j = 0; j < n; ++j) {
8              std::cout << "EECS 281\n";
9          } // for j
10     } // for k
11 } // foo()

```

What is the time complexity of this function in terms of m , n , and p ?

- A) $\Theta(mp)$
- B) $\Theta(np)$
- C) $\Theta(mp+np)$
- D) $\Theta(mnp)$
- E) None of the above

30. Consider the following function implementation:

```

1  int32_t foo(int32_t n) {
2      int32_t count = 0;
3      for (int32_t i = 0; i < n; ++i) {
4          for (int32_t j = n; j > 0; --j) {
5              ++count;
6          } // for j
7      } // for i
8      return count;
9  } // foo()

```

What is the time complexity of this function in terms of n ?

- A) $\Theta(\log(n))$
- B) $\Theta(n)$
- C) $\Theta(n \log(n))$
- D) $\Theta(n^2)$
- E) None of the above

31. Consider the following function implementation:

```

1  int32_t foo(int32_t n) {
2      int32_t count = 0;
3      int32_t root = static_cast<int32_t>(std::floor(std::sqrt(n)));
4      for (int32_t i = n / 2; i < n; ++i) {
5          for (int32_t j = 1; j < n; j *= 2) {
6              for (int32_t k = 0; k < n; k += root) {
7                  ++count;
8              } // for k
9          } // for j
10     } // for i
11     return count;
12 } // foo()

```

What is the time complexity of this function in terms of n ? Note that `std::floor(x)` computes the largest integer value not greater than x , and `std::sqrt(x)` computes the square root of x .

- A) $\Theta(\sqrt{n} \log(n))$
- B) $\Theta(n \log(n))$
- C) $\Theta(n \sqrt{n} \log(n))$
- D) $\Theta(n^2 \log(n))$
- E) $\Theta(n^2 \sqrt{n} \log(n))$

32. Consider the following function implementation:

```

1  int32_t foo(int32_t n) {
2      int32_t count = 0;
3      while (n > 1) {
4          n /= 2;
5          ++count;
6      } // while
7      return count;
8  } // foo()

```

What is the time complexity of this function in terms of n ?

- A) $\Theta(\log(n))$
- B) $\Theta(n)$
- C) $\Theta(n \log(n))$
- D) $\Theta(n^2)$
- E) None of the above

33. Consider the following function implementation:

```

1  void foo(int32_t n) {
2      int32_t k = 0;
3      while (n > 1) {
4          for (int32_t i = 0; i <= n; ++i) {
5              ++k;
6          } // for i
7          n /= 2;
8      } // while
9  } // foo()

```

What is the time complexity of this function in terms of n ? Note: you may find the equation for the sum of a geometric series helpful, where a is the first term and r is the common ratio:

$$S_n = \frac{a(1-r^n)}{1-r}, \text{ where } r \neq 1$$

- A) $\Theta(\log^2(n))$
- B) $\Theta(n)$
- C) $\Theta(n \log(n))$
- D) $\Theta(n^2)$
- E) None of the above

34. Consider the following function implementation:

```

1  void foo(int32_t m, int32_t n) {
2      int32_t k = 0;
3      while (n > m) {
4          for (int32_t i = 0; i < m; i *= 3) {
5              ++k;
6          } // for i
7          n /= 2;
8      } // while
9  } // foo()

```

What is the time complexity of this function in terms of m and n ?

- A) $\Theta(\log(m/n) \log(n))$
- B) $\Theta(\log(n/m) \log(m))$
- C) $\Theta(n \log(m))$
- D) $\Theta(\log(m) \log(n))$
- E) None of the above

35. Consider the following function implementation:

```

1 void foo(int32_t m, int32_t n) {
2     for (int32_t i = 0; i < m; ++i) {
3         for (int32_t j = 1; j < std::log2(n); j *= 4) {
4             for (int32_t k = 1; k < n; k *= 8) {
5                 std::cout << "EECS 281\n";
6             } // for k
7         } // for j
8     } // for i
9 } // foo()

```

What is the time complexity of this function in terms of m and n ?

- A) $\Theta(m \log(n))$
- B) $\Theta(mn \log(n))$
- C) $\Theta(mn^2)$
- D) $\Theta(mn^2 \log(n))$
- E) $\Theta(m \log(n) \log(\log(n)))$

36. Consider the following function, which identifies whether an array has any duplicate values. Let n represent the size of the array.

```

1 bool has_duplicates(int32_t arr[], size_t n) {
2     for (size_t i = 0; i < n; ++i) {
3         for (size_t j = i + 1; j < n; ++j) {
4             if (arr[i] == arr[j]) {
5                 return true;
6             } // if
7         } // for j
8     } // for i
9     return false;
10 } // has_duplicates()

```

What is the best and worst-case time complexity of this function, in terms of n ? Note: you may find the following identity useful:

$$\sum_{x=1}^n x = 1+2+3+4+\dots+n = \frac{n(n+1)}{2}$$

- A) Best-Case: $\Theta(1)$, Worst-Case: $\Theta(n)$
- B) Best-Case: $\Theta(n)$, Worst-Case: $\Theta(n)$
- C) Best-Case: $\Theta(1)$, Worst-Case: $\Theta(n^2)$
- D) Best-Case: $\Theta(n)$, Worst-Case: $\Theta(n^2)$
- E) Best-Case: $\Theta(n^2)$, Worst-Case: $\Theta(n^2)$

37. Consider the following function implementation:

```

1 int32_t foo(int32_t n) {
2     int32_t count = 0;
3     for (int32_t i = 1; i < n; ++i) {
4         for (int32_t j = 1; j < i; ++j) {
5             ++count;
6         } // for j
7     } // for i
8     return count;
9 } // foo()

```

What is the time complexity of the this function in terms of n ? Note: you may find the following identity useful:

$$\sum_{x=1}^n x = 1+2+3+4+\dots+n = \frac{n(n+1)}{2}$$

- A) $\Theta(n)$
- B) $\Theta(n \log(n))$
- C) $\Theta(n^2)$
- D) $\Theta(n^3)$
- E) None of the above

38. Consider the following function implementation:

```

1 int32_t foo(int32_t n) {
2     int32_t count = 0;
3     for (int32_t i = 1; i < n; ++i) {
4         for (int32_t j = 1; j < i; j *= 2) {
5             ++count;
6         } // for j
7     } // for i
8     return count;
9 } // foo()

```

What is the time complexity of the this function in terms of n ?

- A) $\Theta(\log(n))$
- B) $\Theta(n)$
- C) $\Theta(n \log(n))$
- D) $\Theta(n^2)$
- E) None of the above

39. Consider the following function implementation:

```
1  int32_t foo(int32_t n) {
2      int32_t count = 0;
3      for (int32_t i = 1; i < n; i *= 2) {
4          for (int32_t j = 1; j <= i; ++j) {
5              ++count;
6          } // for j
7      } // for i
8      return count;
9  } // foo()
```

What is the time complexity of this function in terms of n ? Note: you may find the equation for the sum of a geometric series helpful.

- A) $\Theta(\log(n))$
- B) $\Theta(n)$
- C) $\Theta(n\log(n))$
- D) $\Theta(n^2)$
- E) None of the above

40. Consider the following function implementation:

```
1  int32_t foo(int32_t n) {
2      int32_t count = 0;
3      for (int32_t i = 1; i <= n; ++i) {
4          for (int32_t j = 1; j * j <= i; ++j) {
5              ++count;
6          } // for j
7      } // for i
8      return count;
9  } // foo()
```

What is the time complexity of this function in terms of n ? Note: you may find the following identity useful:

$$\sum_{x=1}^n x(2x+1) = \frac{1}{6}(4n^3 + 9n^2 + 5n)$$

- A) $\Theta(n\sqrt{n})$
- B) $\Theta(n^2)$
- C) $\Theta(n^2\sqrt{n})$
- D) $\Theta(n^3)$
- E) None of the above

41. Consider the following function implementation:

```
1  int32_t foo(int32_t n) {
2      int32_t count = 0;
3      for (int32_t i = 0; i < n; ++i) {
4          for (int32_t j = n; j > i; j /= 2) {
5              for (int32_t k = j / 2; k < j; ++k) {
6                  for (int32_t m = 1; m < n; m *= 2) {
7                      ++count;
8                  } // for m
9              } // for k
10         } // for j
11     } // for i
12     return count;
13 } // foo()
```

What is the time complexity of this function in terms of n ? Note: you may find the following identity useful:

$$\sum_{x=1}^n \frac{1}{x} = \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n}\right) = \Theta(\log(n))$$

- A) $\Theta(n\log(n))$
- B) $\Theta(n^2\log(n))$
- C) $\Theta(n\log^2(n))$
- D) $\Theta(n^2\log^2(n))$
- E) None of the above

42. Consider the following function implementation:

```
1  int32_t sum(int32_t arr[], size_t n) {
2      int32_t sum = 0;
3      for (int32_t i = 0; i < n; ++i) {
4          sum += arr[i];
5      } // for i
6      return sum;
7  } // foo()
```

What is the *space complexity* and *auxiliary space* used by this function, in terms of the size of `arr`, n ?

- A) Space Complexity: $\Theta(1)$, Auxiliary Space: $\Theta(1)$
- B) Space Complexity: $\Theta(n)$, Auxiliary Space: $\Theta(1)$
- C) Space Complexity: $\Theta(1)$, Auxiliary Space: $\Theta(n)$
- D) Space Complexity: $\Theta(n)$, Auxiliary Space: $\Theta(n)$
- E) None of the above

« Quiz Solutions »

1. **The correct answer is (E).** All of the statements are true. For statement I, pure runtime is an imperfect metric for comparing two algorithms due to the presence of confounding factors like background processes and CPU usage. For statement II, a program's runtime with respect to input size is indeed independent of most external factors, congruent with the claim in statement I. For statement III, when we express runtime in terms of input size, we can more easily categorize and compare them using big-O notation (e.g., it's easier to see that an $\Theta(n)$ algorithm has a lower order of growth than an $\Theta(n^2)$ algorithm).
2. **The correct answer is (A).** None of the statements are true. The asymptotic efficiency of algorithms deals with how runtime increases with the input size as it increases without bound. For instance, as the size of the input grows, we can reason that a $\Theta(n^2)$ algorithm will see its runtime increase much more drastically than a $\Theta(n \log(n))$ algorithm. However, it does not guarantee that a $\Theta(n^2)$ algorithm will always run faster than a $\Theta(n \log(n))$ algorithm, since there are other factors at play (e.g., the algorithm with the larger complexity class may have been run in a best-case scenario or may have better constants and lower-order terms that make it faster for a given input size).
3. **The correct answer is (E).** Big-O represents an asymptotic upper bound, where $a(n)$ is $O(b(n))$ if $b(n)$ serves as an upper bound for $a(n)$. The set of functions that comprise $O(f)$ for a given function f are the set of functions that are bounded above by f . This matches option (E) — for a function to be bounded above by f , it cannot grow quicker than f .
4. **The correct answer is (C).** Lower order terms can be ignored in big-O, and the term with the highest order is $79n$. Coefficients can be ignored as well, so the complexity of *foo* can be simplified to $\Theta(n)$.
5. **The correct answer is (D).** A $\Theta(n^3 \log(n))$ algorithm grows faster than a $\Theta(n^3)$ algorithm due to the presence of an additional $\log(n)$ term that is multiplied with n^3 . A $\Theta(n!)$ algorithm grows faster than a $\Theta(2^n)$ algorithm, and a $\Theta(n^n)$ algorithm grows faster than both a $\Theta(2^n)$ and a $\Theta(n!)$ algorithm. You can also see this by testing a few values of n , as shown below:

n	2	4	8
2^n	4	16	256
$n!$	2	24	40,320
n^n	4	256	16,777,216

6. **The correct answer is (E).** Algorithms involving $\log(n)$ complexities often involve splitting work over and over again and doing a constant number of operations for each split. This is true for both (C) and (D).
7. **The correct answer is (D).** If the input size is too small, the relationship may not be revealed. Plotting out runtimes is most revealing when the input size is large. Note that the number of programs running in parallel is irrelevant since we are talking about asymptotic runtime.
8. **The correct answer is (C).** The program ran better than expected, so it could not have exposed worst-case behavior if all else went to plan.
9. **The correct answer is (A).** None of the statements are true. The best way to disprove these statements is to use trigonometric functions, which do not adhere to many of the rules that are associated with traditional functions. Statement I is false when $f = \sin(x)$ and $g = \cos(x)$. Statement II is false when $f = \sin(x)$, $g = \cos(x)$, and $h = \sin(x) + \cos(x)$. Statement III is false when $f = \sin(x)$, $g = \cos(x)$, and $h = \sin(x)\cos(x)$. Statement IV is false when $f = x^2$ and $g = 1/x$.
10. **The correct answer is (A).** Using log rules, $\log(n^3 9^n)$ can be rewritten as $\log(n^3) + \log(9^n)$. We can move the exponents in front of the log to get $3 \log(n) + n \log(9)$. Both 3 and $\log(9)$ are constants, so they can be ignored, leaving us with $\log(n) + n$. The lower order term of $\log(n)$ can also be dropped, so the final complexity is $\Theta(n)$, not $\Theta(9^n)$. (A) is false. The other options are true. For (B), $\log(n^5 4^{3n})$ can be rewritten as $\log(n^5) + \log(4^{3n}) = 5 \log(n) + 3n \log(4) = \Theta(n)$. For (C), $\log(n!)$ is $\Theta(n \log(n))$ (see example 4.15). For (D), the base of a log does not matter when using big-O notation, so $\log_3(n)$ is $\Theta(\log_2(n))$. For (E), $\log(n^2)$ is $2 \log(n)$ using log rules, which is $\Theta(\log(n))$.
11. **The correct answer is (E).** Asymptotic runtime deals with how an algorithm's runtime scales with respect to input size; the rate of growth of an algorithm is not impacted by the input size, machine performance, or compiler flags.
12. **The correct answer is (A).** The overall time complexity of *foo()* is $\Theta(n^2 + n + n \log(n)) = \Theta(n^2)$, where the runtime of *helper_a()* contributes to the highest order term. Therefore, only improving *helper_b()* or *helper_c()* does not change the overall time complexity of *foo()*. This eliminates options (C) and (D). For option (B), the coefficient does not matter, so the time complexity of *foo()* would still be $\Theta(n^2)$. Only option (A) improves the overall time complexity of the *foo()* function from $\Theta(n^2)$ to $\Theta(n \log(n))$.
13. **The correct answer is (E).** All of statements can be true in the equal case, where $A(n)$, $B(n)$ and $C(n)$ have the same order of growth. Note that big-O, Θ , and Ω can also apply to a tight bound.
14. **The correct answer is (B).** If $A(n)$ is bounded above by $B(n)$ and bounded below by $C(n)$, and all three functions are strictly increasing, then $B(n)$ must also be bounded below by $C(n)$ (otherwise, it could not be an upper bound to $A(n)$, which is also bounded below by $C(n)$). This matches option (B). Note that the other three options are all possible, but only (B) is true in all cases.
15. **The correct answer is (E).** This is not a good experiment for determining which algorithm is the most efficient due to the presence of confounding factors, especially since each algorithm is only timed once. Plus, the conditions of the experiment are not well defined enough to come to a reasonable conclusion (e.g., what input size are you running on, compilation flags used, etc.).
16. **The correct answer is (A).** This graph represents a logarithmic relationship between input size and runtime. None of options (B) through (E) are correct since they all grow at least as fast as linear, which is not true for the relationship in the provided graph.
17. **The correct answer is (D).** Logarithmic functions grow slower than polynomial functions. To make this problem easier to understand, one strategy is to replace $\log(n)$ with another variable, let's say x . We know that $\Theta(\log(x)) < \Theta(x^{1/2}) < \Theta(x) < \Theta(x^2)$. Thus, it must also be true that $\Theta(\log(\log(n))) < \Theta(\log^{1/2}(n)) < \Theta(\log(n)) < \Theta(\log^2(n))$, which matches option (D).
18. **The correct answer is (C).** Options (A) and (B) are incorrect because $\Theta(n^2 \log(n))$ is part of a different complexity class as $\Theta(n^2)$. Option (D) is incorrect because $n^2 \log(n)$ grows faster than n^2 , so n^2 cannot be an upper bound. Only option (C) is correct, since $n^2 + \log(n)$ is $\Theta(n^2)$, which does serve as a lower bound for $n^2 \log(n)$.

19. **The correct answer is (A).** Statement (A) is true because $\log(n) + \log(m) = \log(nm)$, which is an upper bound on $\log(n + m)$. Statement (B) is false because $(n - 1)!$ is a factor of n smaller than $n!$, so it cannot be an upper bound on $n!$. Statement (C) is false because $2^{n+m} = 2^n 2^m$, which is not bounded above by $2^n + 2^m$. Statement (D) is false since sine cannot be strictly bounded by cosine, and vice versa.
20. **The correct answer is (B).** Only II is true. Statement I is false because $f(n) = (2^n)^2 = g(n)^2$, so $f(n)$ and $g(n)$ are not part of the same complexity class. Statement III is false because $e^{n/2}$ and e^n are part of different complexity classes (one is the square root of the other, so they cannot grow at the same rate as input size grows). Statement II is true because $2^{n \log_2(n)} = (2^{\log_2(n)})^n = n^n$.
21. **The correct answer is (C).** If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, that means that $f(n)$ must have a strictly larger rate of growth than $g(n)$. This means that $g(n)$ must be a lower bound on $f(n)$, which matches option (C). Note that you can also use the notation $\omega(g(n))$ to indicate that $f(n)$ is strictly larger than $g(n)$, but you do not need to worry about that here.
22. **The correct answer is (A).** The time to solve a problem with input size S can be expressed as C^S . The time to solve a problem with input size $S + 6$ is C^{S+6} . From this, we know that $T = C^S$ and $9T = C^{S+6}$. Combining these equations, we can conclude that:

$$\frac{9T}{T} = \frac{C^{S+6}}{C^S}$$

$$9 = C^{S+6-S} = C^6$$

$$C = \sqrt[6]{9} = \sqrt[6]{3^2} = \sqrt[3]{3}$$

23. **The correct answer is (C).** The time to solve a problem with input size S can be expressed as $\log_2(S)$. Therefore, the input that can be solved in time $T + 1$ is $1 + \log_2(S)$ (since $T = \log_2(S)$). Using log rules, we can express the following:

$$T + 1 = 1 + \log_2(S)$$

$$T + 1 = \log_2(2) + \log_2(S)$$

$$T + 1 = \log_2(2S)$$

$\log_2(2S)$ is the time needed to solve a problem of input size $2S$. Therefore, we can solve a problem of input size $2S$ in time $T + 1$.

24. **The correct answer is (B).** The time T it takes to solve a problem of input size S is S^2 . Thus, in terms of S , $8T$ can be equivalently expressed as $8S^2$. If we let S' be the size of problems that can be solved in $8S^2$ time, we can solve for S' to get a solution of $(2\sqrt{2})S$.

$$(S')^2 = 8S^2$$

$$S' = \sqrt{8S^2}$$

$$S' = (2\sqrt{2})S$$

25. **The correct answer is (A).** From example 4.15, we know that $\log(n!) = \Theta(n \log(n))$, so $\Theta(n^{280} \log(n!))$ can be rewritten as $\Theta(n^{281} \log(n))$, which grows faster than $\Theta(n^{281})$. $\Theta(281^n) > \Theta(n^{281})$ because exponential algorithms have a higher rate of growth than polynomial ones.
26. **The correct answer is (E).** The limit condition is sufficient to prove that $f(n)$ is $O(g(n))$, but it is not necessary. As a result, it cannot be used to prove that $f(n) \neq O(g(n))$, and the friend is incorrect. In this case, $f(n)$ is indeed $O(g(n))$, since $n \sin(n) + n$ is bounded above by $2n$ as $\sin(n)$ cannot have a value larger than 1.
27. **The correct answer is (C).** The loop from lines 2-4 takes $\Theta(m)$ time, and the loop from lines 6-8 takes $\Theta(n)$ time. These loops are sequential (one happens directly after the other), so their time complexities should be added to get the overall complexity of the `foo()` function — this comes out to $\Theta(m + n)$.
28. **The correct answer is (D).** The loop from lines 3-5 takes $\Theta(n)$ time, and this loop is run m times in the outer loop on line 2. Since this is a nested loop, we multiply $\Theta(n)$ by m to get the overall time complexity of $\Theta(mn)$.
29. **The correct answer is (C).** The loop from lines 3-5 takes $\Theta(m)$ time, and the loop from lines 7-9 takes $\Theta(n)$ time. Since these two loops are sequential, the combined time complexity of lines 3-9 is $\Theta(m + n)$. These two loops are then run p times in an outer loop on line 2, so we multiply $\Theta(m + n)$ by p to get a final time complexity of $\Theta(mp + np)$ for the function `foo()`.
30. **The correct answer is (D).** The inner loop starting on line 4 runs in $\Theta(n)$ time, and it is run n times in the outer loop defined on line 3. All other work in the function takes constant time. Thus, the total time complexity is $n \times \Theta(n) = \Theta(n^2)$.
31. **The correct answer is (C).** The inner loop on line 6 iterates from 0 to n , incrementing by $\sqrt{n}$ each time. This means that $n / \sqrt{n} = \sqrt{n}$ iterations must be run in this innermost loop, each of which performs constant work. The middle loop on line 5 increments from 1 to n , doubling with each iteration — this runs $\log(n)$ iterations. Thus, the loops on line 5 and 6 have a combined time complexity of $\Theta(\sqrt{n} \log(n))$. Lastly, the outermost loop on line 4 runs from $n/2$ to n , which involves $n/2$ iterations. Each of these $n/2$ iterations performs a $\Theta(\sqrt{n} \log(n))$ nested loop, so the overall time complexity of the triple-nested loop is $(n/2) \times \Theta(\sqrt{n} \log(n)) = \Theta(n \sqrt{n} \log(n))$ after dropping all coefficients. Since all other work takes constant time, which is a lower order term that can be dropped in our big-O notation, the overall time complexity of `foo()` is therefore $\Theta(n \sqrt{n} \log(n))$.
32. **The correct answer is (A).** The dominant work within this function can be attributed to the `while` loop on line 3, which divides the input size n by half with every iteration, terminating when n reaches a value less than one. Thus, the time complexity of the loop is $\Theta(\log(n))$.
33. **The correct answer is (B).** There is a loop dependency here, where the number of times the `for` loop runs on line 4 is determined by the value of n , which is being halved within the outer loop defined on line 3. During the first iteration, the loop on line 4 runs n times; during the second iteration, the loop on line 4 runs $n/2$ times, and so on. Since each iteration of the inner loop takes a constant amount of work, the overall time complexity of the nested loop is $n + n/2 + n/4 + \dots = \Theta(n)$ using the formula for a geometric series.
34. **The correct answer is (B).** See example 4.12 for an in-depth explanation on how to solve this problem.
35. **The correct answer is (E).** The inner loop on line 4 runs $\Theta(\log(n))$ times. The middle loop on line 3 runs $\Theta(\log(\log(n)))$ times (since j is multiplied by 4 with each iteration up to $\log_2(n)$). The outer loop on line 2 runs m times. Since this loop is nested, the overall time complexity of the function is the product of these three loop complexities, or $\Theta(m \log(n) \log(\log(n)))$.
36. **The correct answer is (C).** In the best case, the first two numbers in the array are duplicates, which would cause the function to return in constant time (since only two values would be checked). In the worst-case, there are no duplicates, which would require the entire nested loop to run. Using the provided identity, the time complexity in the worst case would be $\Theta(n^2)$.
37. **The correct answer is (C).** There is a loop dependency here, so we will consider both nested `for` loops in tandem. During the first iteration of the outer loop, the inner loop runs 0 times; during the second iteration of the outer loop, the inner loop runs 1 time; during the third iteration of the outer loop, the inner loop runs 2 times, and so on. Using the provided identity, the time complexity of the function comes out to $0 + 1 + 2 + \dots + (n - 1) = \Theta(n^2)$.

38. **The correct answer is (C).** There is a loop dependency here, so we will consider both nested for loops in tandem. The analysis for this problem is similar to the previous problem, with the exception that the amount of work can be approximated as $0 + \log(1) + \log(2) + \log(3) + \dots + \log(n-1)$ instead of $0 + 1 + 2 + 3 + \dots + (n-1)$ because the value of j is doubled instead of incremented within the inner loop. Using log rules, we can conclude that the total work done by the nested loop is $\log(1 \times 2 \times 3 \times \dots \times (n-1)) = \log((n-1)!)$. Furthermore, we know that $\log(n!) = \Theta(n \log(n))$, which allows us to conclude that $\log((n-1)!) = \Theta(\log(n!)) = \Theta(\log(n!) - \log(n)) = \Theta(n \log(n) - \log(n)) = \Theta(n \log(n))$ after dropping lower order terms.

39. **The correct answer is (B).** There is a loop dependency here, so we will consider both nested for loops in tandem. The first iteration of the outer loop runs 1 time, the second iteration of the outer loop runs 2 times, the third iteration of the outer loop runs 4 times, the fourth iteration of the outer loop runs 8 times, and so on. This follows the form of a geometric series, where the iteration count is doubled with each iteration of the outer loop, and the outer loop runs $\log_2(n)$ times. Using the provided summation, we can conclude that:

$$1 + 2 + 4 + 8 + \dots + n = \sum_{k=0}^{\log_2(n)} 2^k = \frac{1(1 - 2^{\log_2(n)+1})}{1-2} = \frac{1-2(2^{\log_2(n)})}{1-2} = \frac{1-2n}{1-2} = \frac{2n-1}{1} = \Theta(n)$$

40. **The correct answer is (A).** There is a loop dependency here, so we will consider both nested loops in tandem. When $i = 1$, the inner loop runs 1 time; when $i = 2$, the inner loop runs 1 time; when $i = 3$, the inner loop runs 1 time; when $i = 4$, the inner loop runs 2 times; and so on. We can express the total number of steps taken by the nested loop using the following summation:

$$1 + 1 + 1 + 2 + 2 + 2 + 2 + 2 + 3 + 3 + 3 + 3 + 3 + 3 + 3 + 3 + 4 + 4 + 4 + \dots + \lfloor \sqrt{n} \rfloor$$

Using the provided identity, we can approximate this as $\sum_{x=1}^{\sqrt{n}} x(2x+1) = \frac{1}{6}(4n^{3/2} + 9n + 5n^{1/2}) = \Theta(n^{3/2}) = \Theta(n\sqrt{n})$

41. **The correct answer is (B).** This function involves a triply-nested loop dependency (where j depends on i , and k depends on j), so we will consider the three loops on lines 3-5 together. Notice that the innermost loop on line 6 always iterates from 1 to n , doubling with each iteration, so the time complexity of this innermost loop is always $\Theta(\log(n))$.

- On the first iteration of the outermost for loop ($i = 0$):
 - The first iteration of the second nested loop ($j = n$) runs the third loop from $k = n/2$ to $k = n$, which performs $\Theta(\log(n))$ work on each iteration. Since the k loop is run $n/2$ times, the total work done on this first iteration is $(n/2)\log(n)$.
 - The second iteration of the j loop runs the k loop from $n/4$ to $n/2$, where $\Theta(\log(n))$ work is performed each iteration. Since the k loop is run $n/4$ times here, the total work done on this second iteration is $(n/4)\log(n)$.
 - The j loop halves j from n to $i = 0$, so the j loop runs $\log(n)$ times during this first iteration of the i loop. Continuing the previous pattern, the total work done on the first iteration of the i loop is $\frac{n}{2}\log(n) + \frac{n}{4}\log(n) + \frac{n}{8}\log(n) + \dots + \frac{n}{2^{\log_2(n)}}\log(n)$.
- On the second iteration of the outermost for loop ($i = 1$):
 - The first iteration of the j loop ($j = n$) runs the k loop $n/2$ times from $k = n$ to $k = n/2$, which performs $\Theta(\log(n))$ work with each iteration. The total work done on this iteration is $(n/2)\log(n)$.
 - The second iteration of the j loop ($j = n/2$) runs the k loop $n/4$ times from $k = n/4$ to $k = n/2$, which performs $\Theta(\log(n))$ work with each iteration. The total work done on this iteration is $(n/4)\log(n)$.
 - The only difference between the second and first iteration of the outer i loop is the number of times the j loop runs. During this iteration, j is halved from n to 1 instead of 0, so the loop runs $\log_2(n-1)$ times instead of $\log_2(n)$ times, which means the total work done on the second iteration of the i loop is $\frac{n}{2}\log(n) + \frac{n}{4}\log(n) + \frac{n}{8}\log(n) + \dots + \frac{n}{2^{\log_2(n-1)}}\log(n)$.
- On the third iteration of the outermost for loop ($i = 2$), the total work done is $\frac{n}{2}\log(n) + \frac{n}{4}\log(n) + \frac{n}{8}\log(n) + \dots + \frac{n}{2^{\log_2(n-2)}}\log(n)$.
- This pattern continues, until the final iteration of the outermost for loop.

Putting this all together, the total work done by the nested loop can be expressed as:

$$\begin{aligned} T(n) &= \left(\frac{n}{2}\log(n) + \frac{n}{4}\log(n) + \dots + \frac{n}{2^{\log_2(n)}}\log(n) \right) + \left(\frac{n}{2}\log(n) + \frac{n}{4}\log(n) + \dots + \frac{n}{2^{\log_2(n-1)}}\log(n) \right) + \left(\dots + \frac{n}{2^{\log_2(1)}}\log(n) \right) \\ &= \frac{1}{2}n\log(n) \left[\left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{2^{\log_2(n)}} \right) + \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{2^{\log_2(n-1)}} \right) + \dots + \left(\dots + \frac{1}{2^{\log_2(1)}} \right) \right] \\ &= \frac{1}{2}n\log(n) \left[\sum_{y=0}^{\log_2(n)} \left(\frac{1}{2} \right)^y + \sum_{y=0}^{\log_2(n-1)} \left(\frac{1}{2} \right)^y + \dots + \sum_{y=0}^{\log_2(1)} \left(\frac{1}{2} \right)^y \right] = \frac{1}{2}n\log(n) \sum_{x=1}^n \left(\sum_{y=0}^{\log_2(x)} \left(\frac{1}{2} \right)^y \right) = \frac{1}{2}n\log(n) \sum_{x=1}^n \left(\frac{1 - \left(\frac{1}{2} \right)^{\log_2(x)+1}}{1 - \frac{1}{2}} \right) \\ &= \frac{1}{2}n\log(n) \sum_{x=1}^n \left(2 \left(1 - \left(\frac{1}{2} \right)^{\log_2(x)+1} \right) \right) = n\log(n) \sum_{x=1}^n \left(1 - \left(\frac{1}{2} \right)^{\log_2(x)+1} \right) = n\log(n) \left(\sum_{x=1}^n 1 - \sum_{x=1}^n \frac{1}{2^{\log_2(x)+1}} \right) = n\log(n) \left(n - \sum_{x=1}^n \frac{1}{x} \right) \\ &= n\log(n) \times (n - \Theta(\log(n))) = n\log(n) \times \Theta(n) = \Theta(n^2 \log(n)) \end{aligned}$$

42. **The correct answer is (B).** The space complexity is the the amount of memory used by an algorithm to solve a problem with respect to input size. On the other hand, auxiliary space only considers the additional memory used by the algorithm but does not include the memory used by the input values themselves. In this example, the `sum()` function acts on an array of size n that is stored in memory, so the total space complexity is $\Theta(n)$. However, this array is an input, so although the function uses its memory, it does not actually allocate it. Beyond the memory passed in as an input, the `sum()` only initializes a constant amount of additional space, so its auxiliary space usage is $\Theta(1)$.